

Java 1.0 — Polymorphism

Java Version: 1.0

Year: 1996

Feature: Polymorphism

UniqueClassName: Polymorphism

1. Definition

Polymorphism means “one name, many forms.”

In Java, polymorphism allows the **same method name or reference** to behave differently depending on the object or context.

The two major forms are:

Type	Meaning	Java mechanism
Compile-time Polymorphism	Method behavior is decided during compilation	Method Overloading
Runtime Polymorphism	Method behavior is decided during execution	Method Overriding + Dynamic Method Dispatch

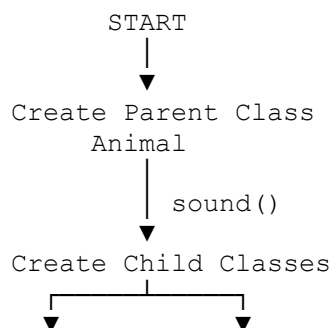
A common example of runtime polymorphism is:

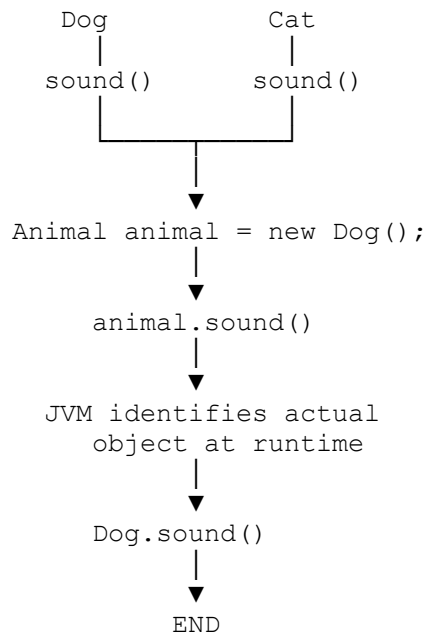
```
Animal animal = new Dog();  
animal.sound();
```

Although the reference type is Animal, the actual object is Dog, so Dog's overridden sound() method executes.

2. Flowchart

Runtime Polymorphism





3. Java Code Example

```
class Animal {

    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {

    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Polymorphism {

    public static void main(String[] args) {

        Animal animal;

        // Parent reference referring to Dog object
```

```
        animal = new Dog();  
        animal.sound();  
  
        // Same parent reference referring to Cat object  
        animal = new Cat();  
        animal.sound();  
    }  
}
```

Output

Dog barks
Cat meows

4. How the Code Works

Animal animal;

A reference variable of type Animal is created.

Then:

```
animal = new Dog();
```

The Animal reference points to a Dog object.

```
animal.sound();
```

At runtime, Java determines that the actual object is Dog, so:

```
Dog.sound()
```

is executed.

Later:

```
animal = new Cat();  
animal.sound();
```

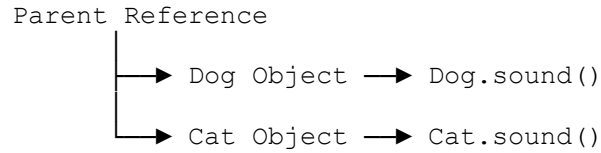
The same animal reference now points to a Cat object, so:

```
Cat.sound()
```

is executed.

This is **runtime polymorphism**.

Key concept



Therefore:

One parent reference → multiple child objects → different method behavior.

Important Java terms

Term	Explanation
Polymorphism	One interface/name with multiple behaviors
Method Overloading	Same method name, different parameter list
Method Overriding	Child class provides its own implementation of a parent method
Upcasting	Assigning a child object to a parent reference
Dynamic Method Dispatch	JVM selects the overridden method at runtime
Parent Reference	Reference whose declared type is the superclass
Actual Object	Object created using new, which determines overridden behavior

Note: Polymorphism was supported in the original Java language, including Java 1.0 (1996); the fundamental mechanism of inheritance, overriding, and dynamic method dispatch is part of Java

's original object-oriented model.

Java 1.0 — Abstract Classes

Java Version: 1.0

Year: 1996

Feature: Abstract Classes

UniqueClassName: AbstractClasses

1. Definition

An **abstract class** in Java is a class declared with the abstract keyword that is intended to serve as a **base class** for other classes.

An abstract class can contain:

- Abstract methods — methods without a body
- Concrete methods — methods with a body
- Variables
- Constructors
- Static methods
- Instance methods
- Static and instance blocks

An abstract class **cannot be instantiated directly**.

```
abstract class Animal {  
    abstract void sound();  
}
```

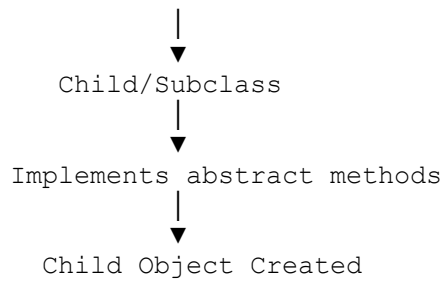
You cannot do:

```
Animal a = new Animal(); // Compilation error
```

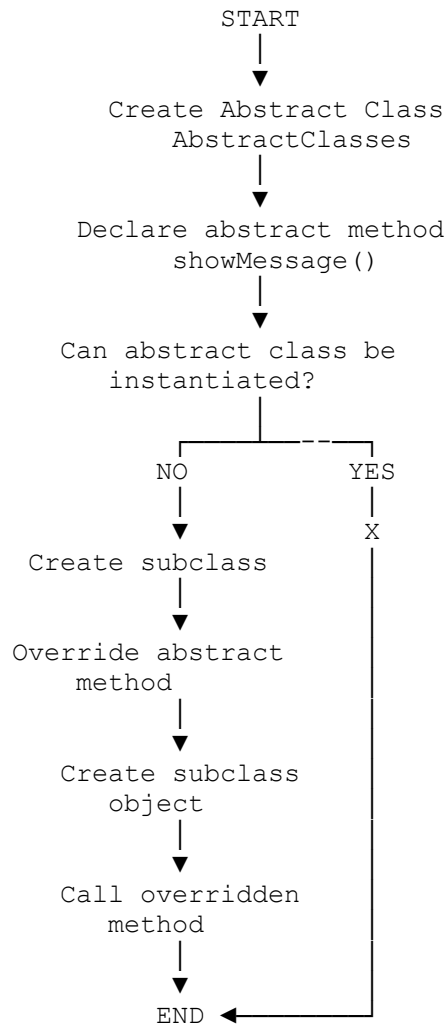
Instead, a subclass must extend the abstract class and provide implementations for its abstract methods.

2. Basic Structure

```
abstract class  
├── Variables  
├── Constructors  
├── Concrete Methods  
└── Abstract Methods
```



3. Flowchart



4. Java Code Example

```
abstract class AbstractClasses {

    // Abstract method
    abstract void showMessage();

    // Concrete method
    void display() {
        System.out.println("This is a concrete method.");
    }
}

class ChildClass extends AbstractClasses {

    // Providing implementation of abstract method
    @Override
    void showMessage() {
        System.out.println("This is the implementation of the abstract
method.");
    }
}

public class Main {

    public static void main(String[] args) {

        // Cannot create object of abstract class directly
        // AbstractClasses obj = new AbstractClasses(); // Error

        // Parent reference pointing to child object
        AbstractClasses obj = new ChildClass();

        // Calling overridden abstract method
        obj.showMessage();

        // Calling concrete method
        obj.display();
    }
}
```

Output

This is the implementation of the abstract method.
This is a concrete method.

5. How the Example Works

Step 1 — Declare an abstract class

```
abstract class AbstractClasses {
```

The abstract keyword tells Java that this class is an abstract class.

Step 2 — Declare an abstract method

```
abstract void showMessage();
```

An abstract method has:

- A method name
- Return type
- Parameters, if required
- **No method body**

```
abstract void showMessage();
```

The subclass is responsible for providing the implementation.

Step 3 — Create a concrete method

```
void display() {  
    System.out.println("This is a concrete method.");  
}
```

An abstract class can contain normal methods with implementations.

Therefore, an abstract class does **not** mean that every method must be abstract.

Step 4 — Extend the abstract class

```
class ChildClass extends AbstractClasses {
```

ChildClass inherits from AbstractClasses.

Because AbstractClasses contains an abstract method, ChildClass must implement it unless ChildClass is also declared abstract.

Step 5 — Override the abstract method

```
@Override
void showMessage() {
    System.out.println("This is the implementation of the abstract method.");
}
```

The child class provides the actual implementation.

Step 6 — Use a parent reference

```
AbstractClasses obj = new ChildClass();
```

This is **upcasting**.

The reference type is:

AbstractClasses

The actual object is:

ChildClass

When this executes:

```
obj.showMessage();
```

Java invokes:

```
ChildClass.showMessage()
```

This also demonstrates **runtime polymorphism**.

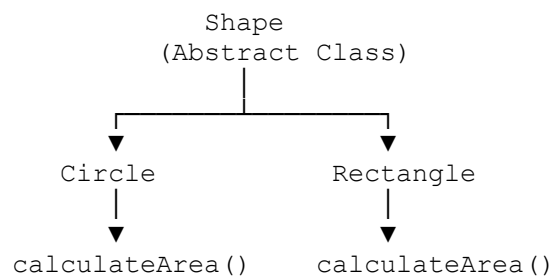
6. Abstract Class vs Concrete Class

Feature	Abstract Class	Concrete Class
abstract keyword	Required	Not required
Direct object creation	✗ Not allowed	✓ Allowed

Abstract methods	✓ Allowed	✗ Not allowed
Concrete methods	✓ Allowed	✓ Allowed
Constructor	✓ Allowed	✓ Allowed
Variables	✓ Allowed	✓ Allowed
Inheritance	✓ Can be extended	✓ Can be extended
Main purpose	Provide a common base/partial implementation	Create complete objects

7. Real-World Example

Consider different shapes:



The general concept of calculating an area belongs to Shape, but the exact calculation differs for each shape.

```

abstract class Shape {

    abstract double calculateArea();

    void display() {
        System.out.println("This is a shape.");
    }
}

class Circle extends Shape {

    private double radius = 5;

    @Override
    double calculateArea() {

```

```

        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {

    private double length = 10;
    private double width = 5;

    @Override
    double calculateArea() {
        return length * width;
    }
}

public class Main {

    public static void main(String[] args) {

        Shape shape;

        shape = new Circle();
        System.out.println("Circle Area: " + shape.calculateArea());

        shape = new Rectangle();
        System.out.println("Rectangle Area: " + shape.calculateArea());
    }
}

```

Output

```

Circle Area      : 78.53981633974483
Rectangle Area   : 50.0

```

Here:

```
Shape shape = new Circle();
```

and:

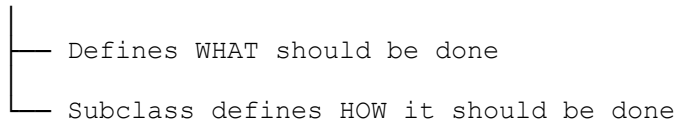
```
shape = new Rectangle();
```

demonstrate **abstraction + inheritance + polymorphism** together.

8. Key Point

The easiest way to remember an abstract class is:

Abstract Class



For example:

```
abstract class Animal {  
    abstract void sound(); // WHAT  
}  
  
class Dog extends Animal {  
    void sound() { // HOW  
        System.out.println("Dog barks");  
    }  
}
```

So, an abstract class provides a common blueprint/base for related classes while allowing subclasses to provide specific implementations.

Java 1.0 — Interfaces

Java Version: 1.0

Year: 1996

Feature: Interfaces

UniqueClassName: Interfaces

1. Definition

An **interface** in Java is a reference type that defines a **contract** that implementing classes must follow.

In the original Java 1.0 model, an interface was primarily used to declare **abstract methods and constants**. A class uses the implements keyword to implement an interface.

```
interface Animal {  
    void sound();  
}
```

A class implements it:

```

class Dog implements Animal {

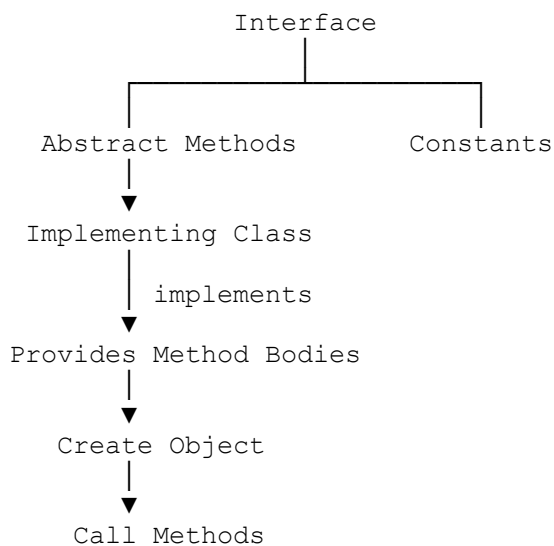
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

```

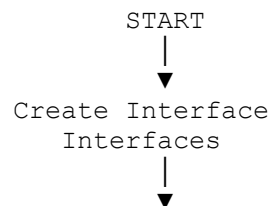
The interface specifies **what must be done**, while the implementing class specifies **how it is done**.

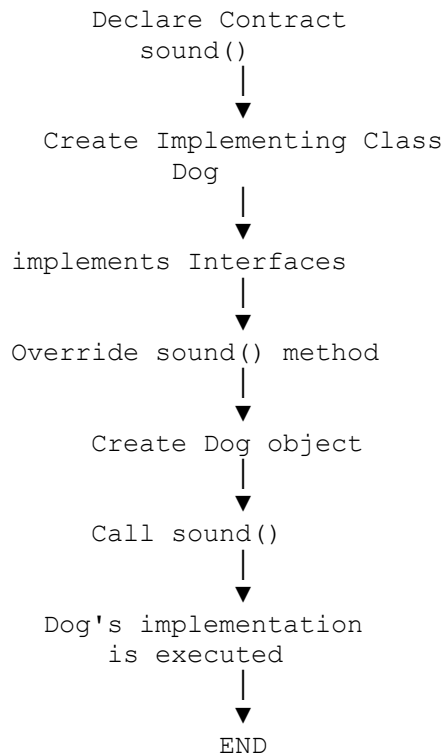
Important: Modern Java interfaces have evolved significantly and can also contain default, static, and private methods. The example above reflects the classic Java 1.0 interface model.

2. Basic Structure



3. Flowchart





4. Java Code Example

```
interface Animal {  
  
    // Abstract method  
    void sound();  
}  
  
class Dog implements Animal {  
  
    @Override  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Interfaces {  
  
    public static void main(String[] args) {  
  
        // Interface reference  
        Animal animal = new Dog();  
  
        // Calling implemented method  
        animal.sound();  
    }  
}
```

```
}
```

Output

Dog barks

5. How the Code Works

Step 1 — Create the interface

```
interface Animal {  
    void sound();  
}
```

Animal defines a contract:

```
Animal  
└── sound()
```

It says that an implementing class must provide a `sound()` method.

Step 2 — Implement the interface

```
class Dog implements Animal {
```

The `implements` keyword establishes the relationship:

```
Dog — implements —> Animal
```

Because `Dog` implements `Animal`, it must provide an implementation of `sound()`.

Step 3 — Implement the method

```
@Override  
public void sound() {  
    System.out.println("Dog barks");  
}
```

The interface declares:

```
void sound();
```

The Dog class provides the implementation:

```
public void sound() {  
    System.out.println("Dog barks");  
}
```

Step 4 — Create an interface reference

```
Animal animal = new Dog();
```

This is a very important Java concept.

The **reference type** is:

Animal

The **actual object** is:

Dog

Therefore:

```
Animal reference  
  |  
  ▼  
Dog object  
  |  
  ▼  
Dog.sound()
```

Step 5 — Invoke the method

```
animal.sound();
```

At runtime, Java identifies that the actual object is a Dog, so:

```
Dog.sound()
```

is executed.

Output:

Dog barks

This demonstrates **interface-based polymorphism**.

6. Multiple Classes Can Implement the Same Interface

One of the major advantages of interfaces is that multiple unrelated classes can implement the same interface.

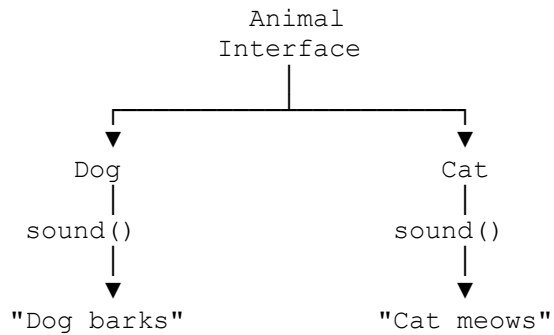
```
interface Animal {  
  
    void sound();  
}  
  
class Dog implements Animal {  
  
    @Override  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat implements Animal {  
  
    @Override  
    public void sound() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class Interfaces {  
  
    public static void main(String[] args) {  
  
        Animal animal;  
  
        animal = new Dog();  
        animal.sound();  
  
        animal = new Cat();  
        animal.sound();  
    }  
}
```

Output

Dog barks

Cat meows

The flow is:



7. Interface and Multiple Inheritance

Java classes cannot extend multiple classes:

// Not allowed

```
class C extends A, B {
}
```

However, a class can implement multiple interfaces:

```
interface Printable {
    void print();
}

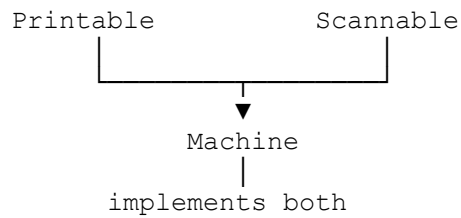
interface Scannable {
    void scan();
}

class Machine implements Printable, Scannable {

    @Override
    public void print() {
        System.out.println("Printing...");
    }

    @Override
    public void scan() {
        System.out.println("Scanning...");
    }
}
```

Relationship:



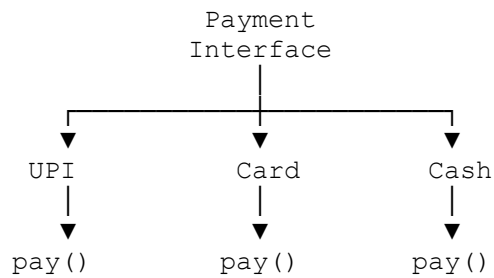
This provides a way for Java to achieve **multiple-type inheritance through interfaces** without allowing a class to inherit implementation from multiple classes.

8. Interface vs Abstract Class

Feature	Interface	Abstract Class
Keyword	interface	abstract class
Implementation relationship	implements	extends
Object creation directly	✗ No	✗ No
Original Java 1.0 abstract methods	✓ Yes	✓ Yes
Original Java 1.0 constants	✓ Yes	✓ Yes
Instance variables	✗ Not as ordinary instance state	✓ Yes
Constructors	✗ No	✓ Yes
Multiple interfaces	✓ A class can implement multiple	✗ A class can extend only one class
Primary purpose	Define a contract/type	Define a common base with shared implementation
Multiple inheritance of type	✓ Yes	✗ No

9. Real-World Example

Consider a payment system:



Java example:

```
interface Payment {  
    void pay(double amount);  
}  
  
class UPI implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using UPI");  
    }  
}  
  
class CreditCard implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using Credit Card");  
    }  
}  
  
public class Interfaces {  
    public static void main(String[] args) {  
        Payment payment;  
  
        payment = new UPI();  
        payment.pay(1000);  
  
        payment = new CreditCard();  
        payment.pay(2000);  
    }  
}
```

```
}  
}
```

Output

Paid ₹1000.0 using UPI
Paid ₹2000.0 using Credit Card

Here, Payment defines **what every payment method must provide**, while UPI and CreditCard define **how the payment is performed**.

10. Key Points to Remember

```
interface  
├── Defines a contract  
├── Implemented using "implements"  
├── Cannot be instantiated directly  
├── Classes provide implementations  
├── Multiple classes can implement it  
└── A class can implement multiple interfaces
```

Simple rule

Interface = WHAT a class must do.
Implementation class = HOW the class does it.

For example:

```
interface Vehicle {  
    void start();           // WHAT  
}  
  
class Car implements Vehicle {  
    public void start() {    // HOW  
        System.out.println("Car starts");  
    }  
}
```

Thus, **interfaces in Java provide a contract-based abstraction mechanism and are one of the fundamental foundations for polymorphism, loose coupling, and multiple inheritance of type.**

Java 1.0 — Exceptions / Exception Handling

Java Version: 1.0

Year: 1996

Feature: Exceptions

UniqueClassName: ExceptionHandling

1. Definition

An **exception** in Java is an abnormal event that occurs during program execution and **disrupts the normal flow of the program.**

Exception handling is Java's mechanism for detecting, handling, and recovering from such exceptional situations instead of allowing the program to terminate unexpectedly.

Java provides keywords such as:

- **try** — contains code that may cause an exception
- **catch** — handles the exception
- **finally** — contains cleanup code that normally executes afterward
- **throw** — explicitly throws an exception
- **throws** — declares exceptions that a method may pass to its caller

Example:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero.");  
}
```

Output:

Cannot divide by zero.

2. Why Exception Handling Is Required

Without exception handling:

```
public class ExceptionHandling {  
  
    public static void main(String[] args) {  
  
        int a = 10;  
        int b = 0;  
  
        int result = a / b;  
  
        System.out.println(result);  
    }  
}
```

The program terminates with an exception such as:

java.lang.ArithmeticException: / by zero

With exception handling:

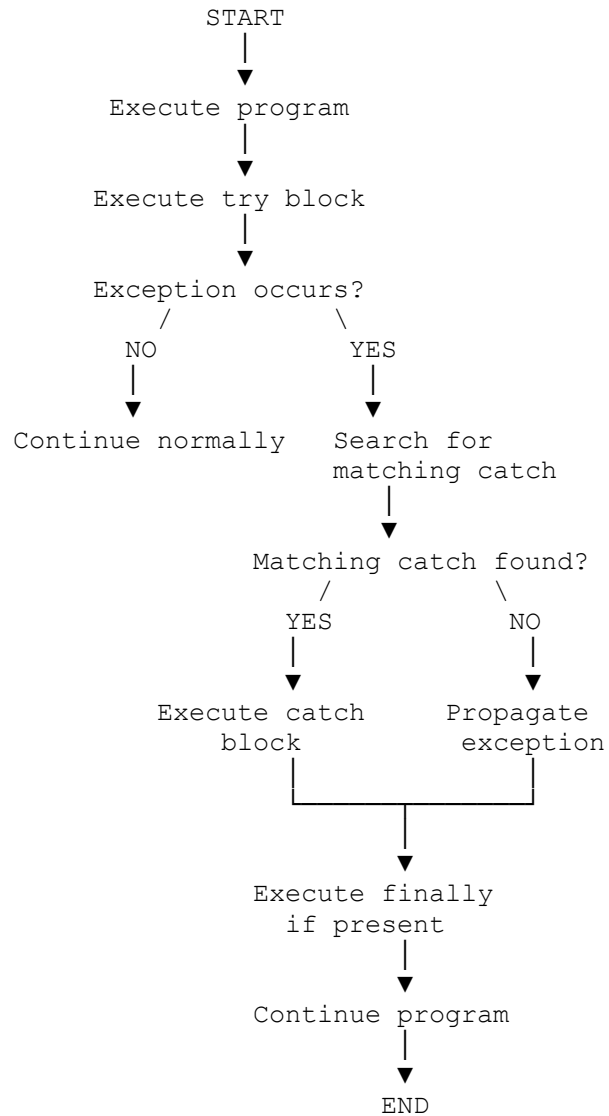
```
public class ExceptionHandling {  
  
    public static void main(String[] args) {  
  
        try {  
            int result = 10 / 0;  
            System.out.println(result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero.");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

Output:

Cannot divide by zero.
Program continues...

The exception was handled, allowing the program to continue.

3. Exception Handling Flowchart



4. Basic Java Code Example

```
public class ExceptionHandling {  
  
    public static void main(String[] args) {  
  
        try {  
            int a = 10;  
            int b = 0;  

```

```
        int result = a / b;

        System.out.println("Result: " + result);
    } catch (ArithmeticException e) {

        System.out.println("Exception handled: "
            + e.getMessage());
    }

    System.out.println("Program continues...");
}
}
```

Output

Exception handled: / by zero
Program continues...

5. How the Code Works

Step 1 — try

```
try {
    int result = 10 / 0;
}
```

The try block contains code that might generate an exception.

Step 2 — Exception occurs

10 / 0

Division by zero is invalid for integer arithmetic.

Java creates an:

ArithmeticException

Step 3 — JVM searches for a handler

Java looks for a catch block capable of handling that exception.

```
catch (ArithmeticException e) {
```

Because the exception is an `ArithmeticException`, this catch block matches.

Step 4 — catch executes

```
System.out.println("Exception handled: "  
    + e.getMessage());
```

The exception is handled.

Step 5 — Program continues

```
System.out.println("Program continues...");
```

Execution continues after the exception-handling structure.

6. try-catch-finally

Java also supports the finally block.

```
public class ExceptionHandling {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 2;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic exception occurred.");  
        } finally {  
            System.out.println("Finally block executed.");  
        }  
    }  
}
```

```
}
```

Output:

Result: 5
Finally block executed.

If an exception occurs:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Exception handled.");  
} finally {  
    System.out.println("Finally block executed.");  
}
```

Output:

Exception handled.
Finally block executed.

The finally block is commonly used for cleanup operations.

7. Multiple catch Blocks

A single try can have multiple catch blocks.

```
public class ExceptionHandling {  
    public static void main(String[] args) {  
        try {  
            int[] numbers = {10, 20, 30};  
            System.out.println(numbers[5]);  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic exception.");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index is invalid.");  
        }  
    }  
}
```

```

        } catch (Exception e) {
            System.out.println("Some other exception occurred.");
        }
    }
}

```

Output:

Array index is invalid.

Java selects the **first matching catch block**.

8. throw Keyword

The throw keyword is used to **explicitly create and throw an exception**.

```

public class ExceptionHandling {
    static void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException(
                "Age must be 18 or above"
            );
        }

        System.out.println("Eligible.");
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Output:

Age must be 18 or above

Flow:

```
checkAge(15)
  ↓
age < 18
  ↓
throw exception
  ↓
catch block
  ↓
Display message
```

9. throws Keyword

The throws keyword is used in a method declaration to indicate that the method may pass an exception to its caller.

```
public class ExceptionHandling {

    static void test() throws Exception {

        throw new Exception("Something went wrong");
    }

    public static void main(String[] args) {

        try {
            test();
        } catch (Exception e) {

            System.out.println(e.getMessage());
        }
    }
}
```

Output:

Something went wrong

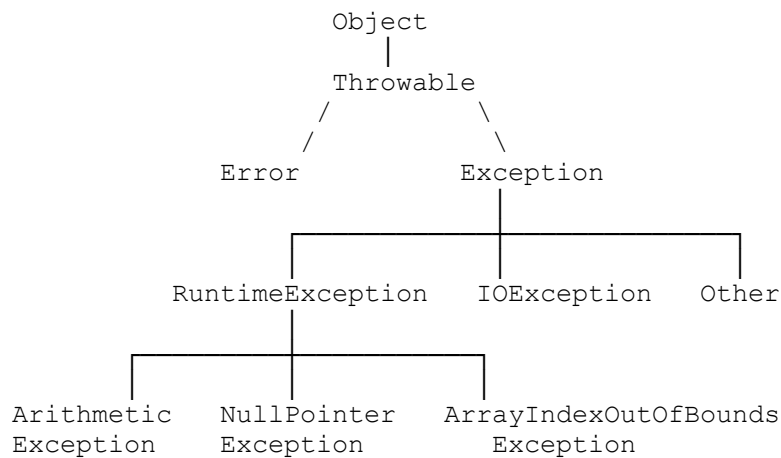
The difference is:

```
throw
|
└─ Actually throws an exception
```

throws
└─ Declares that a method may throw an exception

10. Exception Hierarchy

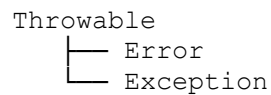
The fundamental hierarchy can be represented as:



Important distinction

Throwable is the root class for objects that can be thrown and caught.

Its two major branches are:



Error generally represents serious JVM/system-level problems, while Exception represents conditions applications commonly handle.

11. Checked vs Unchecked Exceptions

A major distinction in Java is:

Type	Checked by compiler?	Example
------	----------------------	---------

Checked Exception	Yes	IOException
Unchecked Exception	No	ArithmeticException
Runtime Exception	No	NullPointerException

Example of an unchecked exception:

```
int result = 10 / 0;
```

This compiles but fails during execution.

A checked exception generally must be **caught or declared**.

```
void readFile() throws IOException {
    // ...
}
```

12. Real-World Example

Consider a bank withdrawal system:

```
public class ExceptionHandling {

    static void withdraw(double balance, double amount) {

        if (amount > balance) {
            throw new IllegalArgumentException(
                "Insufficient balance"
            );
        }

        System.out.println("Withdrawal successful.");
        System.out.println("Remaining balance: "
            + (balance - amount));
    }

    public static void main(String[] args) {

        try {

            withdraw(5000, 7000);

        } catch (IllegalArgumentException e) {

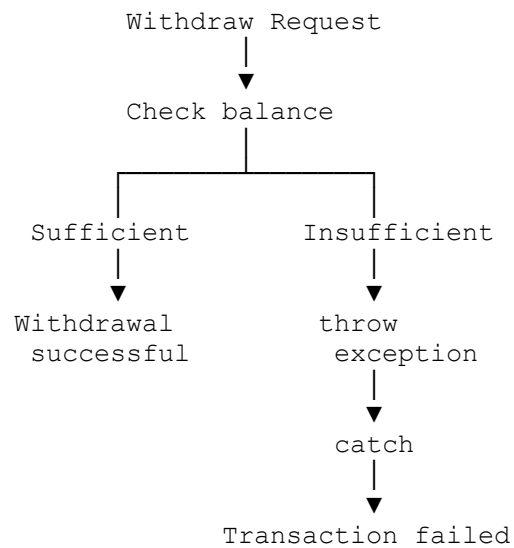
            System.out.println("Transaction failed: "
                + e.getMessage());
        }
    }
}
```

```
}  
    }  
}
```

Output:

Transaction failed: Insufficient balance

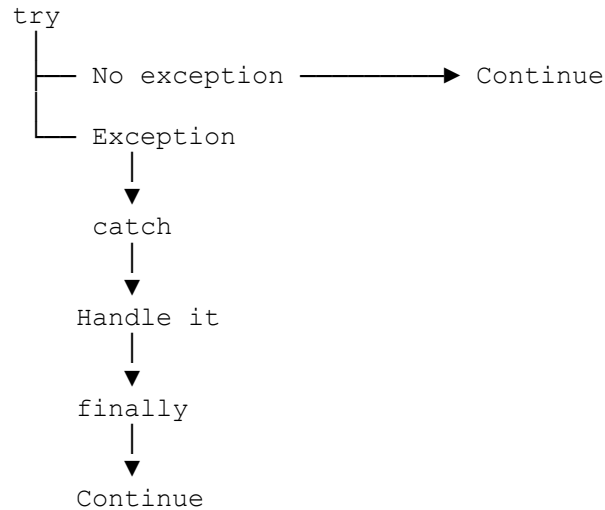
The flow is:



13. Key Exception Handling Keywords

Keyword	Purpose
try	Defines code that may produce an exception
catch	Handles an exception
finally	Performs code that should execute after handling/attempt
throw	Explicitly throws an exception
throws	Declares exceptions a method may

14. Simple Formula to Remember



In one sentence

Exception handling in Java allows a program to detect and handle abnormal runtime conditions using mechanisms such as try, catch, finally, throw, and throws, thereby separating error-handling logic from normal program logic.

Java 1.0 note: Exception handling, Throwable, Exception, Error, try, catch, finally, throw, and throws were part of Java's original language design.

Java 1.0 — Thread Basics

Java Version: 1.0

Year: 1996

Feature: Threads Basics

UniqueClassName: ThreadBasics

1. Definition

A **thread** is a lightweight unit of execution within a program.

Multithreading means executing multiple threads concurrently within the same Java application.

For example, an application could have:

- One thread performing calculations

- Another thread reading data
- Another thread handling user interaction

Java has supported threads from its original Java 1.0 release.

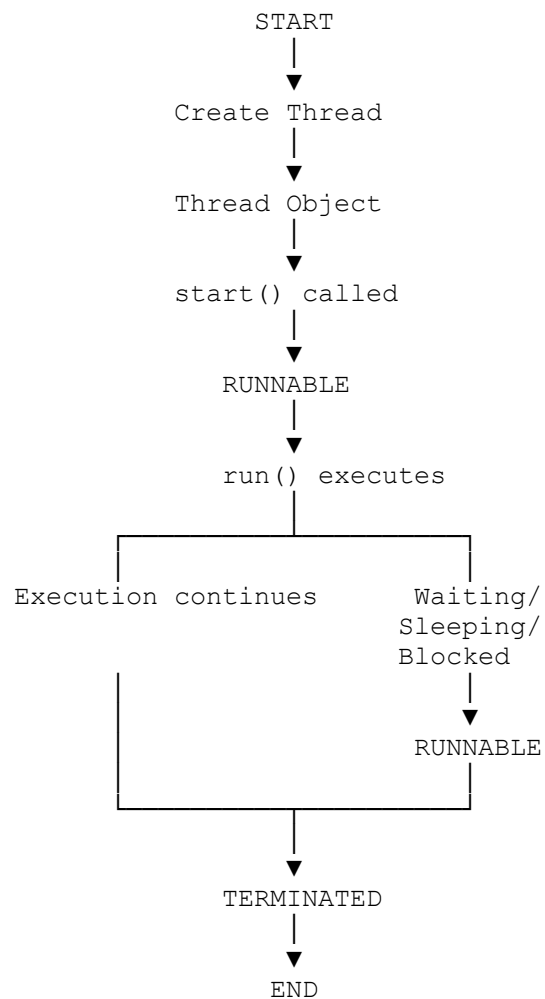
The central class is:

`java.lang.Thread`

A thread can be created by:

1. **Extending Thread**
 2. **Implementing Runnable**
-

2. Thread Lifecycle — Basic Flowchart



Important

Calling:

```
thread.start();
```

causes the JVM to arrange for the thread's `run()` method to execute.

Calling:

```
thread.run();
```

directly **does not start a new thread**; it is simply a normal method invocation.

3. Java Code Example — Extending Thread

```
class MyThread extends Thread {  
  
    @Override  
    public void run() {  
  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(  
                "Child Thread: " + i  
            );  
        }  
    }  
}  
  
public class ThreadBasics {  
  
    public static void main(String[] args) {  
  
        MyThread thread = new MyThread();  
  
        thread.start();  
  
        System.out.println(  
            "Main thread is running..."  
        );  
    }  
}
```

Possible Output

Because threads execute concurrently, the exact order is **not guaranteed**.

```
Main thread is running...
Child Thread: 1
Child Thread: 2
Child Thread: 3
Child Thread: 4
Child Thread: 5
```

Another valid execution could be:

```
Child Thread: 1
Child Thread: 2
Main thread is running...
Child Thread: 3
Child Thread: 4
Child Thread: 5
```

This is because the JVM's thread scheduler determines when each runnable thread gets execution time.

4. How the Example Works

Step 1 — Create a thread class

```
class MyThread extends Thread {
```

MyThread inherits functionality from Java's Thread class.

```
Thread
  |
  ▼
MyThread
```

Step 2 — Override run()

```
@Override
public void run() {
    // Thread's task
}
```

The run() method contains the work that the thread should perform.

For example:

```
public void run() {  
    for (int i = 1; i <= 5; i++) {  
        System.out.println("Child Thread: " + i);  
    }  
}
```

Step 3 — Create the thread object

```
MyThread thread = new MyThread();
```

At this point, the thread object exists, but the new thread of execution has **not yet started**.

Step 4 — Start the thread

```
thread.start();
```

This is the critical operation.

```
thread.start()  
  ↓  
JVM schedules thread  
  ↓  
run()  
  ↓  
Thread executes
```

5. Main Thread

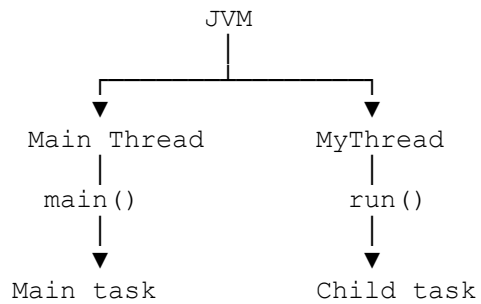
Every Java application begins with a thread called the **main thread**.

When this executes:

```
public static void main(String[] args)
```

the JVM invokes `main()` on the main thread.

Therefore, in the example there are at least two threads:



6. Example Showing Thread Names

```
class MyThread extends Thread {

    @Override
    public void run() {

        System.out.println(
            "Running thread: "
            + Thread.currentThread().getName()
        );
    }
}

public class ThreadBasics {

    public static void main(String[] args) {

        System.out.println(
            "Current thread: "
            + Thread.currentThread().getName()
        );

        MyThread thread = new MyThread();

        thread.start();
    }
}
```

Possible output:

Current thread: main
Running thread: Thread-0

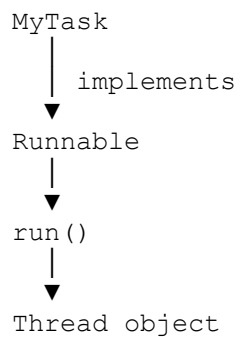
The exact automatically assigned thread name can vary.

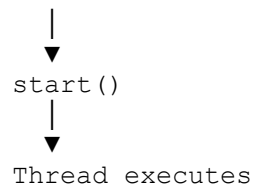
7. Creating a Thread Using Runnable

Another fundamental approach is implementing Runnable.

```
class MyTask implements Runnable {  
  
    @Override  
    public void run() {  
  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(  
                "Runnable Thread: " + i  
            );  
        }  
    }  
}  
  
public class ThreadBasics {  
  
    public static void main(String[] args) {  
  
        MyTask task = new MyTask();  
  
        Thread thread = new Thread(task);  
  
        thread.start();  
    }  
}
```

Flow:





8. Thread vs Runnable

Approach	Syntax	Main idea
Extend Thread	class MyThread extends Thread	Thread class itself represents the task
Implement Runnable	class MyTask implements Runnable	Separates the task from the thread
Start	thread.start()	Starts concurrent execution
Task method	run()	Contains thread task

Runnable is often preferred when the class already needs to extend another class, because Java supports single class inheritance.

For example:

```
class MyClass extends SomeOtherClass
    implements Runnable {

    public void run() {
        // task
    }
}
```

9. start() vs run()

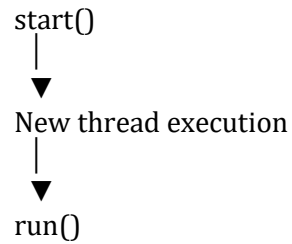
This is one of the most important thread concepts.

Correct

```
Thread t = new MyThread();
```

`t.start();`

Conceptually:

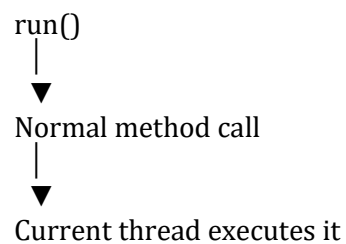


Not equivalent

`Thread t = new MyThread();`

`t.run();`

Here:

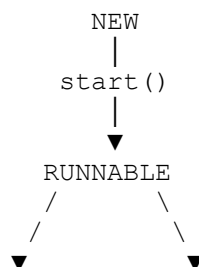


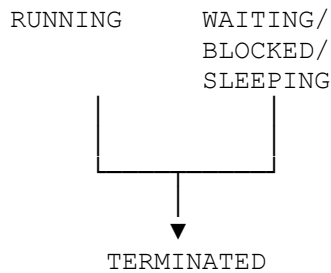
So:

`start()` starts a new thread; `run()` contains the task and, when invoked directly, does not create a new thread.

10. Basic Thread States

A simplified view is:





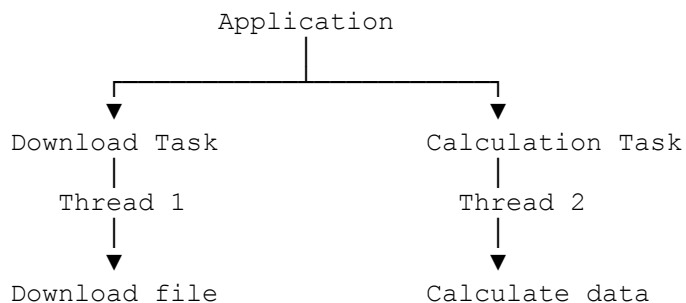
Modern Java's official thread-state model uses:

NEW
 RUNNABLE
 BLOCKED
 WAITING
 TIMED_WAITING
 TERMINATED

The distinction between "running" and "runnable" is important: Java's Thread.State enum does **not** have a separate RUNNING state; running is encompassed by RUNNABLE.

11. Simple Real-World Example

Imagine an application performing two independent tasks:



Java example:

```

class DownloadTask implements Runnable {

    @Override
    public void run() {

        for (int i = 1; i <= 3; i++) {
            System.out.println(
                "Downloading part " + i
            );
        }
    }
}
  
```

```

}

class CalculationTask implements Runnable {

    @Override
    public void run() {

        for (int i = 1; i <= 3; i++) {
            System.out.println(
                "Calculating part " + i
            );
        }
    }
}

public class ThreadBasics {

    public static void main(String[] args) {

        Thread download =
            new Thread(new DownloadTask());

        Thread calculation =
            new Thread(new CalculationTask());

        download.start();
        calculation.start();

    }
}

```

Possible output:

```

Downloading part 1
Calculating part 1
Downloading part 2
Calculating part 2
Calculating part 3
Downloading part 3

```

The ordering can vary from execution to execution.

12. Key Points to Remember

```

Thread
|
|— Lightweight unit of execution
|
|— Thread class → java.lang.Thread

```

- Task is normally placed in run()
- start() begins thread execution
- run() directly = ordinary method call
- main() executes on the main thread
- Multiple threads can execute concurrently

One-line definition

A thread in Java is an independent path of execution within a program, and multithreading allows multiple such execution paths to operate concurrently.

Java 1.0 note: Basic thread support, including Thread, Runnable, start(), run(), and the core synchronization model, was part of Java's original 1.0 platform. Modern Java has since added richer concurrency facilities such as executors, futures, fork/join, and virtual threads.

Java 1.0 — Synchronization

Java Version: 1.0

Year: 1996

Feature: Synchronization

UniqueClassName: SynchronizationBasics

1. Definition

Synchronization in Java is a mechanism used to control access to a **shared resource** when multiple threads execute concurrently.

Its primary purpose is to prevent **race conditions** and maintain **data consistency**.

Java provides the synchronized keyword for synchronization.

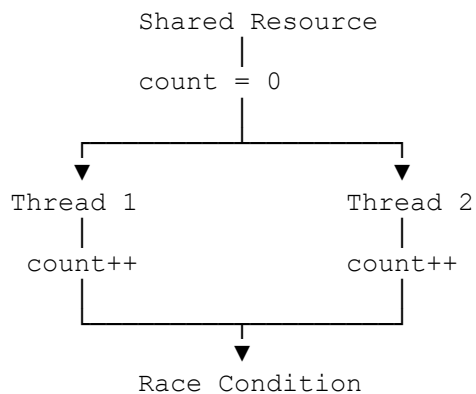
For example:

```
synchronized void increment() {  
    count++;  
}
```

When a thread enters a synchronized method or block, it acquires the object's **monitor lock**. Other threads attempting to enter synchronized code protected by the same monitor must wait until the lock is released.

2. Why Synchronization Is Required

Suppose two threads simultaneously modify the same variable:



The operation:

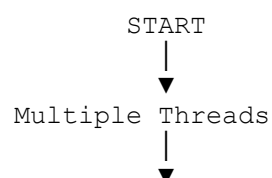
`count++;`

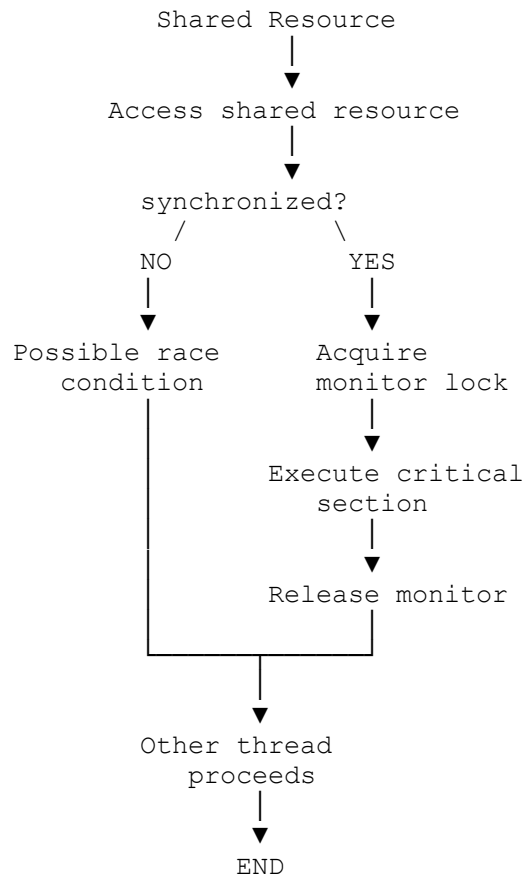
is not one indivisible operation. Conceptually, it involves:

Read count
↓
Add 1
↓
Write count

Two threads can interfere with one another if this operation is performed without proper synchronization.

3. Synchronization Flowchart





4. Basic Java Code Example

```
class Counter {  
    private int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
  
    int getCount() {  
        return count;  
    }  
}  
  
public class SynchronizationBasics {  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Counter counter = new Counter();
```

```

Thread thread1 = new Thread(() -> {
    for (int i = 0; i < 1000; i++) {
        counter.increment();
    }
});

Thread thread2 = new Thread(() -> {
    for (int i = 0; i < 1000; i++) {
        counter.increment();
    }
});

thread1.start();
thread2.start();

thread1.join();
thread2.join();

System.out.println("Final count: "
    + counter.getCount());
}
}

```

Output

Final count: 2000

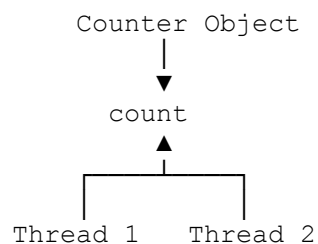
Because `increment()` is synchronized, only one thread at a time can execute that synchronized method on the same counter object.

5. How the Code Works

Step 1 — Shared resource

```
private int count = 0;
```

count is shared by both threads.

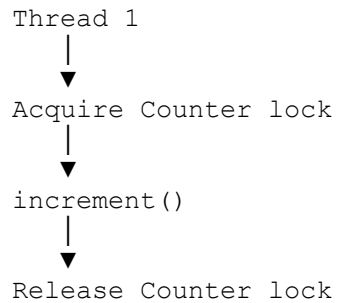


Step 2 — Synchronized method

```
synchronized void increment() {  
    count++;  
}
```

The synchronized keyword protects the method.

Conceptually:



If Thread 2 tries to enter the same synchronized method on the same Counter object while Thread 1 holds the monitor, Thread 2 must wait.

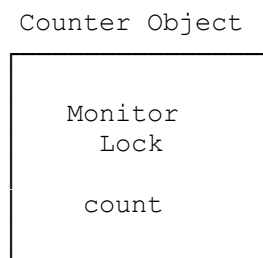
6. Monitor Lock

Every Java object can be associated with an intrinsic **monitor**.

For:

```
Counter counter = new Counter();
```

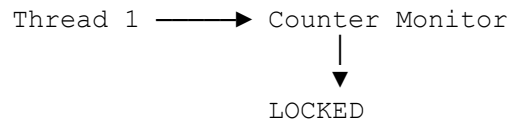
conceptually:



When Thread 1 enters:

```
counter.increment();
```

it acquires the monitor associated with counter.



Thread 2 attempting the same synchronized method must wait until Thread 1 exits.

7. Synchronization With a Block

Synchronization does not have to cover an entire method.

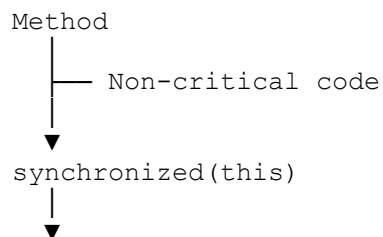
You can synchronize only a specific section of code.

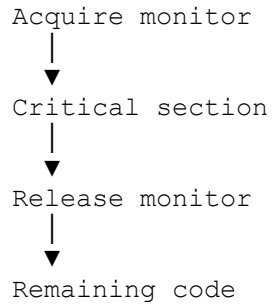
```
class Counter {  
    private int count = 0;  
    void increment() {  
        synchronized (this) {  
            count++;  
        }  
    }  
    int getCount() {  
        return count;  
    }  
}
```

The syntax is:

```
synchronized (object) {  
    // critical section  
}
```

Flow:





8. Synchronized Method vs Synchronized Block

Feature	Synchronized Method	Synchronized Block
Syntax	synchronized void method()	synchronized(lock) {}
Lock	Object's monitor for instance method	Specified object's monitor
Scope	Entire method	Only selected section
Control	Less granular	More granular
Typical use	Entire method must be protected	Only critical section needs protection

Example:

Synchronized method

```
synchronized void increment() {  
    count++;  
}
```

Synchronized block

```
void increment() {  
  
    synchronized (this) {  
        count++;  
    }  
}
```

For an instance method synchronized on this, these two forms have essentially the same locking scope for that method's execution.

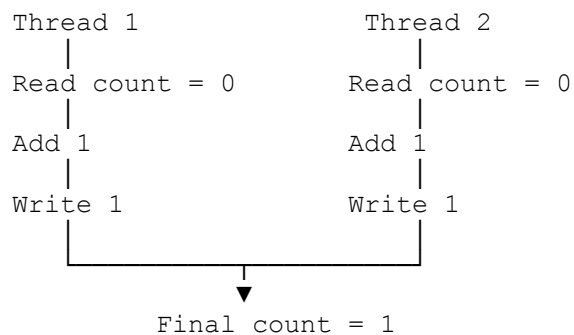
9. Race Condition Without Synchronization

Consider:

```
class Counter {  
    private int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

Two threads can execute:

Initial count = 0



The expected result after two increments is:

2

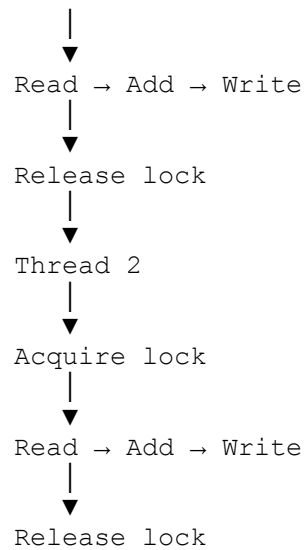
but the result can be:

1

This is a **lost update** caused by a race condition.

With synchronization:

```
Thread 1  
  |  
  ▼  
Acquire lock
```



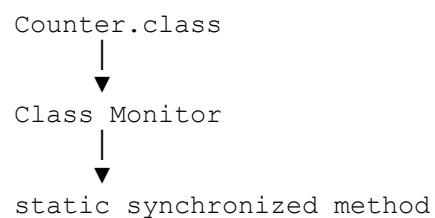
The updates are serialized.

10. Static Synchronization

A static synchronized method locks the **Class object**, rather than an individual instance.

```
class Counter {  
    private static int count = 0;  
  
    static synchronized void increment() {  
        count++;  
    }  
  
    static int getCount() {  
        return count;  
    }  
}
```

Conceptually:



This is different from:

```
synchronized void increment()
```

which locks the individual object instance.

11. Instance vs Static Synchronization

Type	Lock associated with
synchronized instance method	Current object (this)
synchronized(this)	Current object
synchronized static method	Class object
synchronized(MyClass.class)	MyClass.class

For example:

```
synchronized void method() {  
}
```

is conceptually similar to:

```
void method() {  
    synchronized (this) {  
        // method body  
    }  
}
```

Whereas:

```
static synchronized void method() {  
}
```

is conceptually similar to:

```
static void method() {  
    synchronized (MyClass.class) {  
        // method body  
    }  
}
```

12. Real-World Example — Bank Account

Synchronization is useful when multiple threads access a shared bank account.

```
class BankAccount {
    private int balance = 1000;

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(
                Thread.currentThread().getName()
                + " is withdrawing " + amount
            );

            balance -= amount;

            System.out.println(
                "Remaining balance: " + balance
            );
        } else {
            System.out.println(
                "Insufficient balance"
            );
        }
    }
}

public class SynchronizationBasics {
    public static void main(String[] args)
        throws InterruptedException {

        BankAccount account = new BankAccount();

        Thread t1 = new Thread(
            () -> account.withdraw(700),
            "Thread-1"
        );

        Thread t2 = new Thread(
            () -> account.withdraw(700),
            "Thread-2"
        );

        t1.start();
        t2.start();
    }
}
```

```
        t1.join();
        t2.join();
    }
}
```

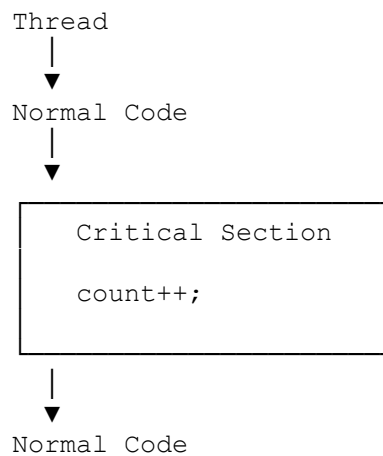
Possible output:

Thread-1 is withdrawing 700
Remaining balance: 300
Insufficient balance

Without synchronization, both threads could potentially check the balance before either performs the withdrawal, creating an incorrect result.

13. Important Concept: Critical Section

A **critical section** is a portion of code that accesses shared mutable data and therefore requires controlled concurrent access.



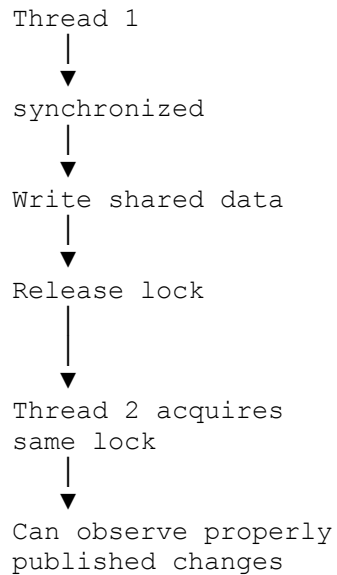
Synchronization protects this critical section.

14. Synchronization Does Two Important Things

Synchronization is not only about preventing simultaneous access.

It also provides **memory-visibility and ordering guarantees** between threads when the same monitor is used correctly.

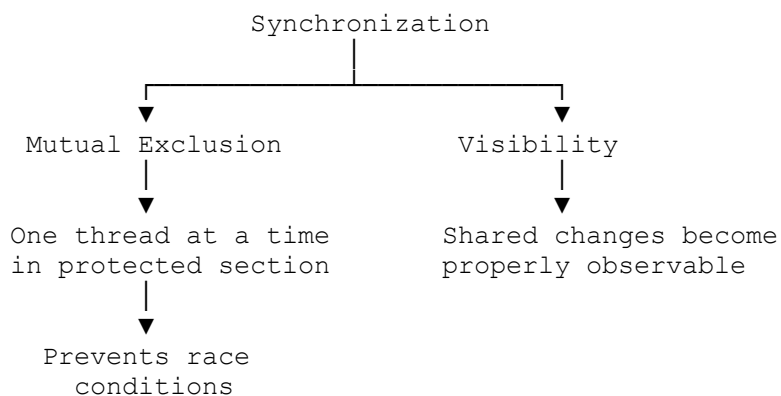
Conceptually:



Thus synchronization helps with:

1. **Mutual exclusion** — preventing multiple threads from entering the protected critical section simultaneously.
 2. **Memory visibility** — establishing happens-before relationships between unlock and subsequent lock operations on the same monitor.
-

15. Key Points to Remember



Important keywords/concepts

Concept	Meaning
synchronized	Protects code using a monitor
Monitor	Intrinsic lock associated with an object/class
Critical section	Code accessing shared mutable state
Race condition	Incorrect behavior caused by uncontrolled concurrent access
Instance synchronization	Locks an object instance
Static synchronization	Locks the corresponding Class object
Synchronized block	Synchronizes a selected section
Mutual exclusion	Only one thread enters a protected monitor region at a time

One-line definition

Synchronization in Java is the mechanism for coordinating concurrent threads so that access to shared resources is controlled, preventing race conditions and providing the necessary memory-visibility guarantees.

Java 1.0 note: The synchronized keyword and Java's intrinsic monitor-based synchronization model were part of Java's original concurrency model in Java 1.0 (1996).

Java 1.0 — Wait/Notify

Java Version: 1.0

Year: 1996

Feature: wait() / notify()

UniqueClassName: WaitNotify

1. Definition

Java's **wait() / notify() mechanism** allows threads to **coordinate with each other** when they share a common object or resource.

It is based on the methods:

wait()
notify()
notifyAll()

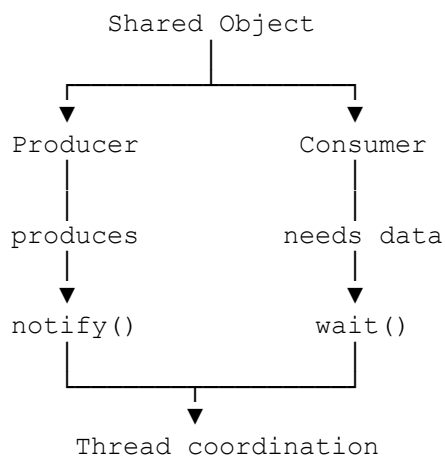
These methods are defined in the java.lang.Object class, so **every Java object can act as a monitor used for thread communication.**

Purpose

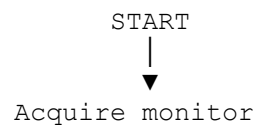
- wait() — causes the current thread to wait and **releases the monitor** it holds.
- notify() — wakes **one** thread waiting on the same object's monitor.
- notifyAll() — wakes **all** threads waiting on the same object's monitor.

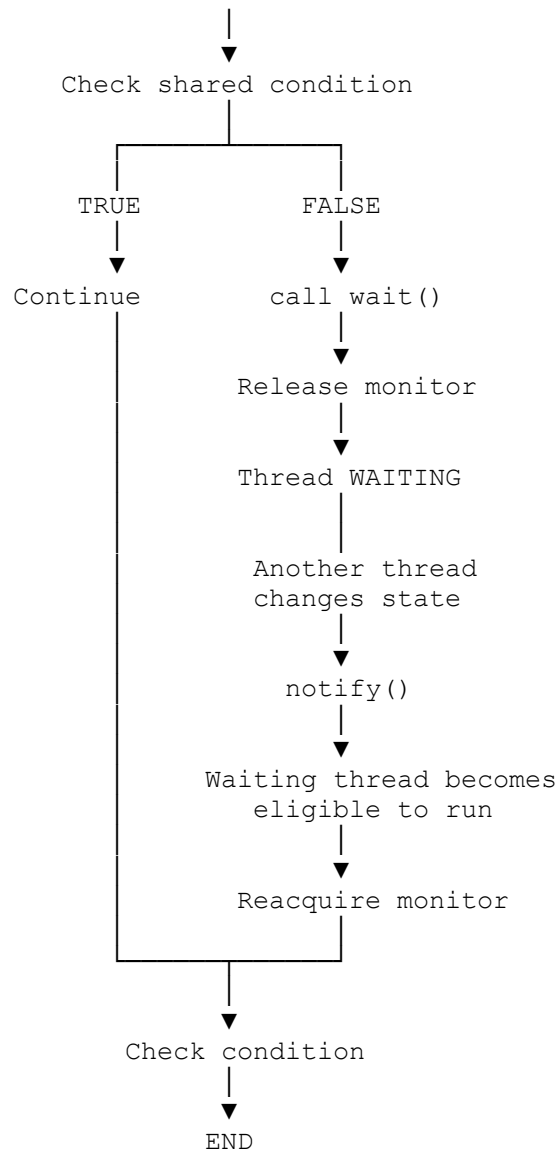
These methods must be called while the current thread **owns that object's monitor**, normally inside a synchronized method or synchronized block.

2. Basic Concept



3. Flowchart





Important: notify() does not directly transfer the lock to the awakened thread. The notifying thread continues until it exits the synchronized region (or otherwise releases the monitor); the awakened thread must then reacquire the monitor before wait() returns.

4. Java Code Example

A simple producer-consumer example demonstrates wait() and notify().

```
class SharedData {  
    private int value;
```

```

private boolean available = false;

synchronized void produce(int value) {

    while (available) {
        try {
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }

    this.value = value;
    available = true;

    System.out.println(
        "Produced: " + value
    );

    notify();
}

synchronized void consume() {

    while (!available) {
        try {
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }

    System.out.println(
        "Consumed: " + value
    );

    available = false;

    notify();
}

}

public class WaitNotify {

    public static void main(String[] args)
        throws InterruptedException {

        SharedData data = new SharedData();

        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                data.produce(i);
            }
        });

```

```

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                data.consume();
            }
        });

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();
    }
}

```

Possible Output

```

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

```

The exact ordering can vary depending on thread scheduling, but the condition logic ensures that the consumer does not consume unavailable data.

5. How wait() Works

Suppose the consumer executes:

```

while (!available) {
    wait();
}

```

If available is false:

```

Consumer
  ↓
available == false
  ↓
wait()
  ↓
Release monitor

```

↓
Thread enters WAITING

This is different from simply sleeping.

wait():

1. Must be called while holding the object's monitor.
 2. Releases that monitor.
 3. Places the thread in the waiting set for that object.
 4. Eventually allows another thread to acquire the monitor and change the condition.
 5. After notification and reacquiring the monitor, the waiting thread returns from wait() and checks the condition again.
-

6. How notify() Works

The producer executes:

```
available = true;  
notify();
```

Conceptually:

```
Producer  
↓  
Produce data  
↓  
available = true  
↓  
notify()  
↓  
One waiting thread  
becomes eligible  
↓  
Producer eventually  
releases monitor  
↓  
Consumer reacquires monitor  
↓  
wait() returns
```

Again, `notify()` **does not release the monitor immediately**.

7. `notify()` vs `notifyAll()`

`notify()`

`notify();`

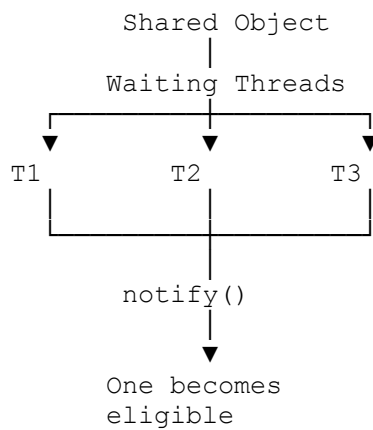
Wakes **one** thread from the object's waiting set.

`notifyAll()`

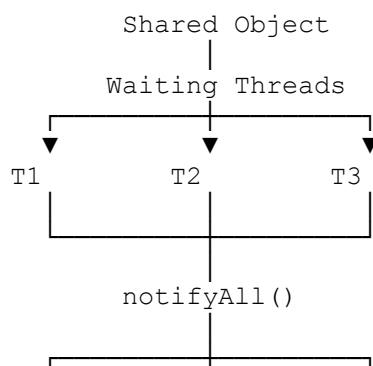
`notifyAll();`

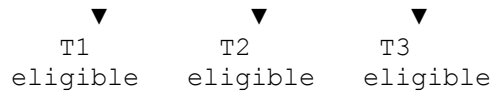
Wakes **all** threads waiting on that object's monitor.

Conceptually:



Whereas:





They still have to compete to reacquire the monitor.

8. Why wait() Is Used Inside while

You will commonly see:

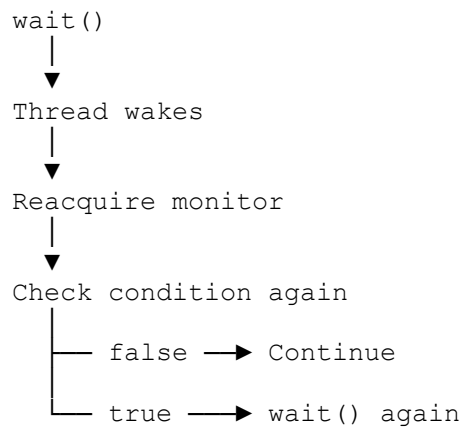
```
while (!available) {  
    wait();  
}
```

rather than:

```
if (!available) {  
    wait();  
}
```

The while loop is important because after wait() returns, the condition must be **checked again**.

For example:



This is the standard guarded-block pattern.

9. wait() vs sleep()

This is a very important distinction.

Feature	wait()	sleep()
Defined in	Object	Thread
Releases monitor?	✓ Yes	✗ No
Purpose	Thread coordination	Pause execution
Requires synchronized context?	✓ Yes, for the object's monitor	✗ No
Woken by	Notification/interrupt/other permitted wakeup	Timeout/interrupt
Typical use	Producer-consumer coordination	Delay/timing

Example:

```
wait();
```

means:

"I cannot proceed until the condition changes; release this monitor and wait."

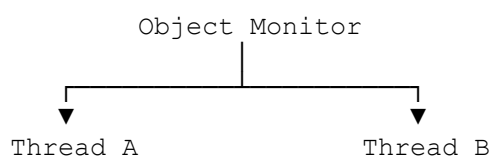
Whereas:

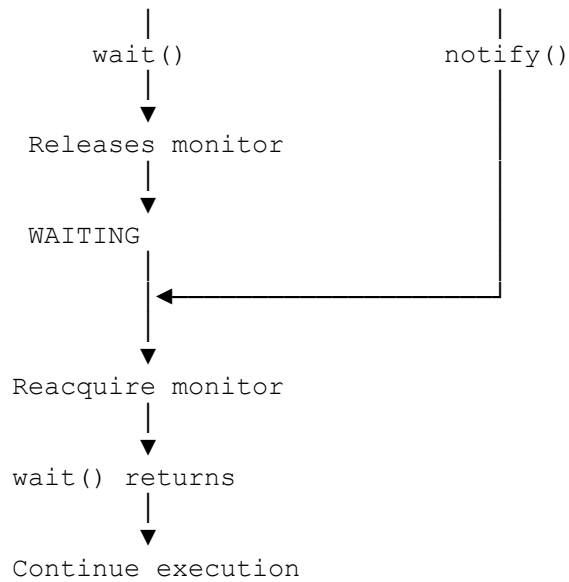
```
Thread.sleep(1000);
```

means:

"Pause this thread for approximately one second."

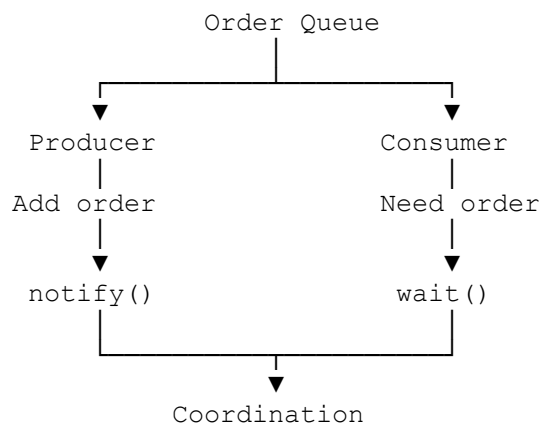
10. wait() / notify() Relationship





11. Real-World Example — Producer and Consumer

Imagine an online order system.



For example:

```
class OrderQueue {  
    private String order;  
    private boolean available;  
  
    synchronized void placeOrder(String order) {  
        while (available) {  
            try {
```

```

        wait();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return;
    }
}

this.order = order;
available = true;

System.out.println(
    "Order placed: " + order
);

notify();
}

synchronized void processOrder() {

    while (!available) {
        try {
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }

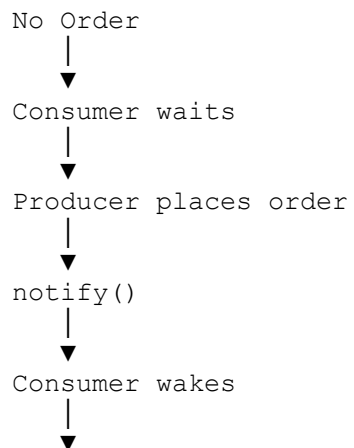
    System.out.println(
        "Order processed: " + order
    );

    available = false;

    notify();
}
}

```

The important idea is:



12. Important Rules

Rule 1 — Call on the correct object

```
synchronized (lock) {  
    lock.wait();  
}
```

The object used for wait() should correspond to the monitor being held.

Rule 2 — Must own the monitor

This is invalid:

```
Object lock = new Object();
```

```
lock.wait(); // IllegalMonitorStateException
```

because the current thread does not own lock's monitor.

Correct:

```
Object lock = new Object();
```

```
synchronized (lock) {  
    lock.wait();  
}
```

Similarly:

```
synchronized (lock) {  
    lock.notify();  
}
```

Rule 3 — Always protect the condition

A good pattern is:

```
synchronized (lock) {  
    while (!condition) {  
        lock.wait();  
    }  
  
    // Work with the condition satisfied  
}
```

And the modifying thread:

```
synchronized (lock) {  
  
    // Change shared state  
  
    lock.notify();  
}
```

13. IllegalMonitorStateException

If a thread calls wait(), notify(), or notifyAll() without owning the object's monitor, Java throws:

IllegalMonitorStateException

For example:

```
Object lock = new Object();
```

```
lock.notify(); // Error at runtime
```

Correct:

```
synchronized (lock) {  
    lock.notify();  
}
```

14. Key Concepts

Method/Concept	Meaning
----------------	---------

wait()	Current thread waits and releases the object's monitor
notify()	Makes one waiting thread eligible to resume
notifyAll()	Makes all waiting threads eligible to resume
Object	Declares these methods
Monitor	Intrinsic lock associated with an object
Waiting	Thread waits for a condition/notification
synchronized	Required to own the monitor when using these methods
while	Rechecks the condition after waking
IllegalMonitorStateException	Thrown when monitor ownership rules are violated

15. Simple Formula to Remember

```
wait()
=
"Condition is not satisfied.
Release the lock and wait."

notify()
=
"Something changed.
One waiting thread may proceed."

notifyAll()
=
"Something changed.
All waiting threads may proceed."
```

One-line definition

wait()/notify() is Java's monitor-based inter-thread communication mechanism in which a thread can wait for a condition while releasing a monitor, and another thread can signal waiting threads after changing the shared state.

Java 1.0 note: wait(), notify(), notifyAll(), intrinsic monitors, and the synchronized mechanism were part of Java's original thread coordination model in Java 1.0 (1996). Modern Java also provides higher-level concurrency utilities that are often preferable for new applications.

Java 1.0 — Arrays

Java Version: 1.0

Year: 1996

Feature: Arrays

UniqueClassName: Arrays

1. Definition

An **array** in Java is an object that stores a **fixed-size, ordered collection of elements of the same type**.

Each element is accessed using an **index**, and Java array indexes start from **0**.

For example:

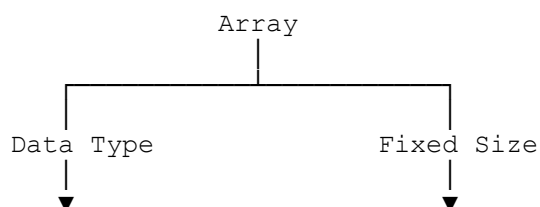
```
int[] numbers = {10, 20, 30, 40, 50};
```

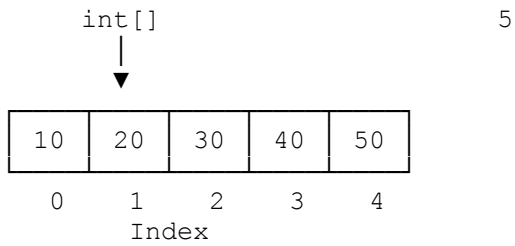
Conceptually:

Index:	0	1	2	3	4
Value:	10	20	30	40	50

The size of an array is fixed when the array object is created.

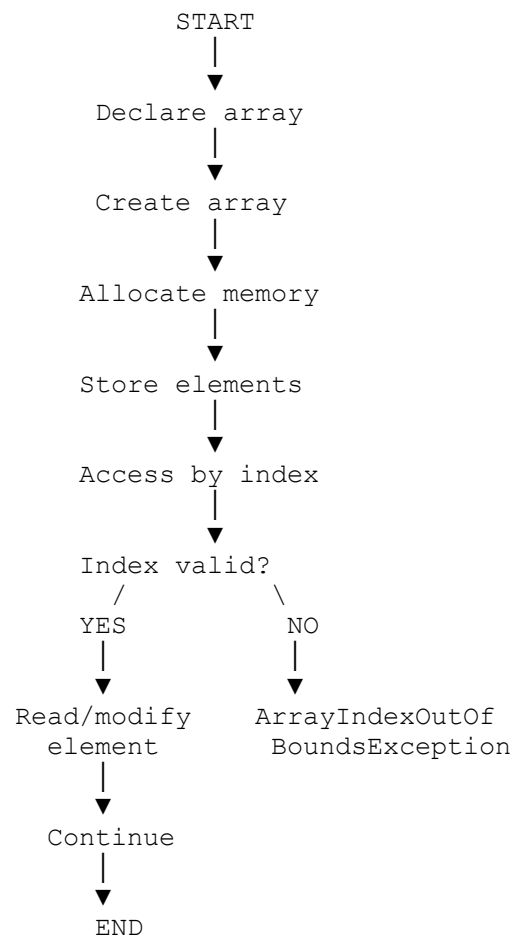
2. Basic Array Structure





3. Flowchart

Creating and Accessing an Array



4. Java Code Example

```
public class Arrays {  
  
    public static void main(String[] args) {  
  
        // Create and initialize an array  
        int[] numbers = {10, 20, 30, 40, 50};  
  
        // Access individual elements  
        System.out.println("First element: " + numbers[0]);  
        System.out.println("Third element: " + numbers[2]);  
        System.out.println("Fifth element: " + numbers[4]);  
  
        // Print all elements  
        System.out.println("All elements:");  
  
        for (int i = 0; i < numbers.length; i++) {  
            System.out.println(numbers[i]);  
        }  
    }  
}
```

Output

```
First element: 10  
Third element: 30  
Fifth element: 50  
All elements:  
10  
20  
30  
40  
50
```

5. How the Code Works

Step 1 — Declare an array

```
int[] numbers;
```

This declares a reference variable capable of referring to an integer array.

You can also write:

```
int numbers[];
```

Both forms are valid.

The preferred modern style is:

```
int[] numbers;
```

Step 2 — Create the array

```
numbers = new int[5];
```

This creates an array containing five int elements.

Index:	0	1	2	3	4
Value:	0	0	0	0	0

The default value of an int array element is 0.

Step 3 — Assign values

```
numbers[0] = 10;  
numbers[1] = 20;  
numbers[2] = 30;  
numbers[3] = 40;  
numbers[4] = 50;
```

The array becomes:

10	20	30	40	50
0	1	2	3	4

6. Declaration, Creation and Initialization

These are three related but distinct operations.

Declaration

```
int[] numbers;
```

Creation

```
numbers = new int[5];
```

Initialization

```
numbers[0] = 10;  
numbers[1] = 20;
```

They can also be combined:

```
int[] numbers = new int[5];
```

Or initialized directly:

```
int[] numbers = {10, 20, 30, 40, 50};
```

7. Array Indexing

Java arrays use **zero-based indexing**.

For:

```
int[] numbers = {10, 20, 30, 40, 50};
```

the relationship is:

Element		Index
-----		-----
10	→	0
20	→	1
30	→	2
40	→	3
50	→	4

Therefore:

```
numbers[0]    // 10  
numbers[2]    // 30  
numbers[4]    // 50
```

The last valid index is:

`array.length - 1`

So for an array of length 5:

Last index = 5 - 1 = 4

8. The length Property

Every Java array has a length field.

```
int[] numbers = {10, 20, 30, 40, 50};  
System.out.println(numbers.length);
```

Output:

5

Important:

numbers.length

is an **array property**, not a method.

Therefore:

```
numbers.length // Correct  
numbers.length() // Incorrect
```

9. Iterating Through an Array

Traditional for loop

```
public class Arrays {  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40, 50};  
        for (int i = 0; i < numbers.length; i++) {  
            System.out.println(  
                "Index " + i + ": " + numbers[i]  
            );  
        }  
    }  
}
```

Output:

```
Index 0: 10
Index 1: 20
Index 2: 30
Index 3: 40
Index 4: 50
```

10. Enhanced for Loop

Java also provides the enhanced for loop:

```
int[] numbers = {10, 20, 30, 40, 50};

for (int number : numbers) {
    System.out.println(number);
}
```

Output:

```
10
20
30
40
50
```

Conceptually:

Array

```

|
├── 10 ──> number
├── 20 ──> number
├── 30 ──> number
├── 40 ──> number
└── 50 ──> number
```

11. Primitive Array

An array can store primitive values.

```
int[] numbers = {10, 20, 30};

double[] prices = {10.5, 20.5, 30.5};
```

```
char[] letters = {'A', 'B', 'C'};

boolean[] flags = {true, false, true};
```

12. Object Array

An array can also contain references to objects.

```
class Student {

    String name;

    Student(String name) {
        this.name = name;
    }
}

public class Arrays {

    public static void main(String[] args) {

        Student[] students = new Student[3];

        students[0] = new Student("Rahul");
        students[1] = new Student("Amit");
        students[2] = new Student("Priya");

        for (Student student : students) {
            System.out.println(student.name);
        }
    }
}
```

Output:

Rahul
Amit
Priya

Conceptually:

```
Student[]
|
├── [0] → Student("Rahul")
├── [1] → Student("Amit")
└── [2] → Student("Priya")
```

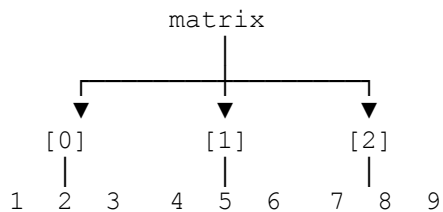
13. Multidimensional Arrays

Java supports arrays whose elements are themselves arrays.

For example:

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

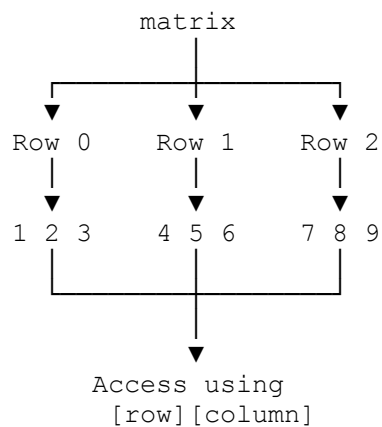
Conceptually:



Access:

```
matrix[0][0]    // 1  
matrix[1][2]    // 6  
matrix[2][1]    // 8
```

14. Two-Dimensional Array Flow



Example:

```
public class Arrays {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
  
        for (int row = 0; row < matrix.length; row++) {  
            for (int column = 0;  
                column < matrix[row].length;  
                column++) {  
                System.out.print(  
                    matrix[row][column] + " "  
                );  
            }  
            System.out.println();  
        }  
    }  
}
```

Output:

```
1 2 3  
4 5 6  
7 8 9
```

15. Array Index Out of Bounds

If an array has five elements:

```
int[] numbers = {10, 20, 30, 40, 50};
```

valid indexes are:

```
0 1 2 3 4
```

This is invalid:

```
System.out.println(numbers[5]);
```

because index 5 does not exist.

Java throws:

`ArrayIndexOutOfBoundsException`

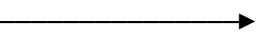
Conceptually:

Array length = 5

Valid:

0 - 1 - 2 - 3 - 4

Invalid:

5  `ArrayIndexOutOfBoundsException`

16. Default Values of Array Elements

When an array is created using `new`, its elements receive default values.

Type	Default value
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
boolean	false
Reference types	null

Example:

```
int[] numbers = new int[3];
```

```
System.out.println(numbers[0]);
```

```
System.out.println(numbers[1]);  
System.out.println(numbers[2]);
```

Output:

0
0
0

17. Important Properties of Java Arrays

Property	Description
Fixed size	Size cannot be changed after creation
Zero-based	First index is 0
Same element type	All elements have the declared array component type
Object	Arrays are objects in Java
length	Provides the number of elements
Bounds checking	Java checks array indexes at runtime
Primitive support	Can store primitive values
Object support	Can store object references
Multidimensional	Can create arrays of arrays
Type-safe	Array assignments are checked by Java's type system

18. Array vs Collection

Feature	Array	Collection
Size	Fixed	Usually dynamically resizable
Index access	Direct	Depends on collection
Primitive elements	Yes	Collections store objects, with boxing for primitives
length	Yes	No
Generic type parameters	No	Yes
Example	int[]	ArrayList<Integer>
Introduced in Java	Java 1.0	Collection Framework later, Java 1.2

19. Real-World Example — Student Marks

```
public class Arrays {  
  
    public static void main(String[] args) {  
  
        int[] marks = {  
            85, 90, 78, 92, 88  
        };  
  
        int total = 0;  
  
        for (int mark : marks) {  
            total += mark;  
        }  
  
        double average =  
            (double) total / marks.length;  
  
        System.out.println("Total: " + total);  
        System.out.println("Average: " + average);  
    }  
}
```

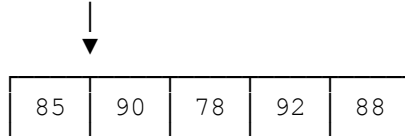
```
}
```

Output:

Total : 433
Average : 86.6

Flow:

Student Marks

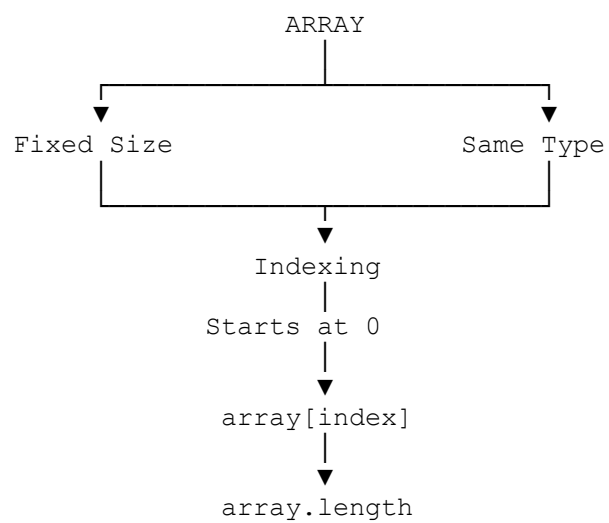


Calculate total

Calculate average

Display result

20. Key Points to Remember



Simple definition

An array in Java is a fixed-size object that stores an ordered collection of elements of a specified component type, with each element accessed through a zero-based index.

Java 1.0 note: Array syntax and the fundamental Java array model were part of the original Java 1.0 language. The separate `java.util.Arrays` utility class should not be confused with the array language feature; `java.util.Arrays` was introduced later with the Java 1.2 Collections-era libraries.

Java 1.0 — Strings

Java Version: 1.0

Year: 1996

Feature: Strings

UniqueClassName: `StringBasics`

1. Definition

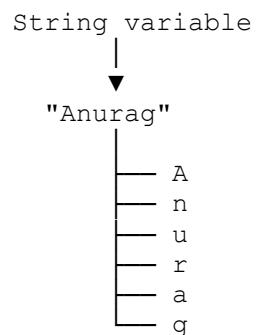
A **String** in Java is an object that represents a sequence of characters.

Java provides the `java.lang.String` class for working with text.

Example:

```
String name = "Anurag";
```

Conceptually:



Important characteristic

Java String objects are **immutable**.

This means that once a String object has been created, its character contents cannot be changed. Operations that appear to modify a string actually create or return another String object.

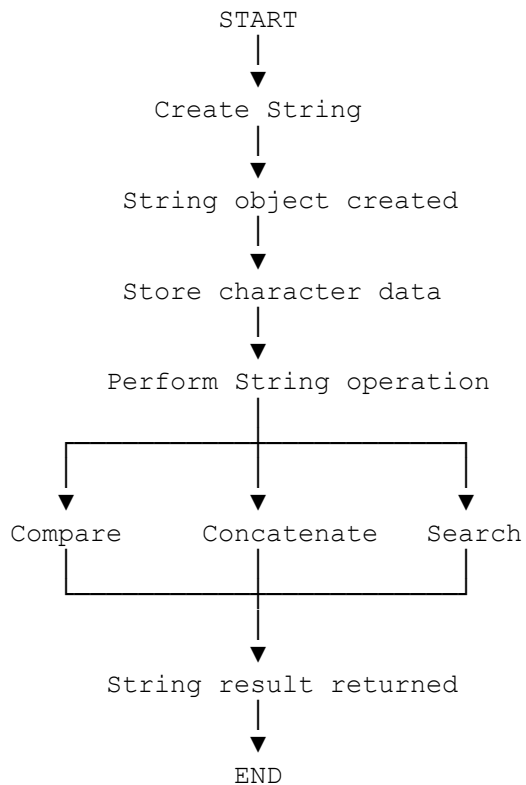
For example:

```
String text = "Hello";
```

```
text = text + " Java";
```

The original "Hello" string is not modified; a new string representing "Hello Java" is produced and assigned to text.

2. String Flowchart



3. Basic Java Code Example

```
public class StringBasics {
```

```
public static void main(String[] args) {  
  
    String firstName = "Anurag";  
    String lastName = "Bharti";  
  
    String fullName = firstName + " " + lastName;  
  
    System.out.println("First Name: " + firstName);  
    System.out.println("Last Name: " + lastName);  
    System.out.println("Full Name: " + fullName);  
    System.out.println("Length: " + fullName.length());  
}  
}
```

Output

First Name:	Anurag
Last Name:	Bharti
Full Name:	Anurag Bharti
Length:	13

4. How the Code Works

Step 1 — Create a String

```
String firstName = "Anurag";
```

This creates a String representing the characters:

```
A n u r a g  
0 1 2 3 4 5
```

Step 2 — Create another String

```
String lastName = "Bharti";
```

Step 3 — Concatenate Strings

```
String fullName = firstName + " " + lastName;
```

The + operator concatenates strings.

Conceptually:

```
"Anurag"  
  +  
" "  
  +  
"Bharti"  
  |  
  ▼  
"Anurag Bharti"
```

Step 4 — Get String length

```
fullName.length()
```

length() returns the number of char values in the string.

```
A n u r a g   B h a r t i  
0 1 2 3 4 5 6 7 8 9 10 11 12
```

Therefore:

```
length = 13
```

5. String Creation

There are two common ways to create strings.

String literal

```
String text = "Hello";
```

Using new

```
String text = new String("Hello");
```

Both produce a String representing "Hello".

However, string literals are handled specially by the JVM through the **String pool**, whereas `new String(...)` explicitly creates a new String object.

6. String Pool

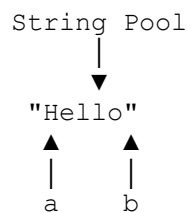
Java maintains a pool of interned strings.

Consider:

```
String a = "Hello";  
String b = "Hello";
```

The two variables can refer to the same pooled "Hello" object.

Conceptually:

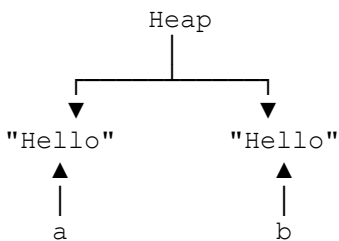


But:

```
String a = new String("Hello");  
String b = new String("Hello");
```

creates distinct String objects.

Conceptually:



7. == vs equals()

This is one of the most important String concepts.

==

For objects, == compares **references**, not String contents.

```
String a = "Hello";  
String b = "Hello";
```

```
System.out.println(a == b);
```

Typically:

true

because both literals can refer to the same interned string.

But:

```
String a = new String("Hello");  
String b = new String("Hello");
```

```
System.out.println(a == b);
```

prints:

false

because the references point to different objects.

equals()

```
System.out.println(a.equals(b));
```

prints:

true

because String.equals() compares the actual character contents.

Rule

== → reference identity
equals() → String content

8. Common String Methods

Java's String class provides many operations.

Method	Purpose	Example
length()	Returns length	"Hello".length() → 5
charAt()	Gets character at index	"Hello".charAt(1) → e
substring()	Extracts part of string	"Hello".substring(1,4) → ell
equals()	Compares contents	"A".equals("A")
equalsIgnoreCase()	Case-insensitive comparison	"java".equalsIgnoreCase("JAVA")
concat()	Concatenates strings	"Hello".concat(" Java")
indexOf()	Finds position	"Hello".indexOf('e') → 1
lastIndexOf()	Finds last occurrence	"Hello".lastIndexOf('l') → 3
startsWith()	Tests beginning	"Java".startsWith("Ja")
endsWith()	Tests ending	"Java".endsWith("va")
toUpperCase()	Converts to uppercase	"java".toUpperCase()
toLowerCase()	Converts to lowercase	"JAVA".toLowerCase()
trim()	Removes leading/trailing whitespace	" Java ".trim()
replace()	Replaces characters/sequences	"Java".replace('a','o')

9. Java Code Using String Methods

```
public class StringBasics {
```

```
public static void main(String[] args) {  
  
    String text = "Hello Java";  
  
    System.out.println("String: " + text);  
    System.out.println("Length: " + text.length());  
    System.out.println("Character at index 1: "  
        + text.charAt(1));  
  
    System.out.println("Uppercase: "  
        + text.toUpperCase());  
  
    System.out.println("Lowercase: "  
        + text.toLowerCase());  
  
    System.out.println("Contains Java: "  
        + text.contains("Java"));  
  
    System.out.println("Starts with Hello: "  
        + text.startsWith("Hello"));  
  
    System.out.println("Ends with Java: "  
        + text.endsWith("Java"));  
    }  
}
```

Output

```
String: Hello Java  
Length: 10  
Character at index 1: e  
Uppercase: HELLO JAVA  
Lowercase: hello java  
Contains Java: true  
Starts with Hello: true  
Ends with Java: true
```

10. String Immutability

Consider:

```
String text = "Hello";  
  
text.concat(" Java");  
  
System.out.println(text);
```

Output:

Hello

Why?

Because `concat()` returns a new `String`; it does not modify the original string.

Correct:

```
text = text.concat(" Java");
```

```
System.out.println(text);
```

Output:

Hello Java

Flow:

```
text
|
▼
"Hello"
|
concat(" Java")
▼
New String
"Hello Java"
|
▼
text references new String
```

11. String Concatenation

Strings can be concatenated using `+`.

```
String first = "Hello";
```

```
String second = "World";
```

```
String result = first + " " + second;
```

```
System.out.println(result);
```

Output:

Hello World

The `+` operator is overloaded for string concatenation.

For example:

```
int age = 30;
```

```
String result = "Age: " + age;
```

```
System.out.println(result);
```

Output:

Age: 30

When one operand is a String, Java performs string concatenation.

12. String and Character Indexing

Consider:

```
String text = "JAVA";
```

Its indexes are:

Character:	J	A	V	A
Index:	0	1	2	3

Therefore:

```
text.charAt(0); // J
text.charAt(1); // A
text.charAt(2); // V
text.charAt(3); // A
```

An invalid index:

```
text.charAt(4);
```

causes:

StringIndexOutOfBoundsException

13. String vs char[]

A String represents text:

```
String text = "Java";
```

A character array stores individual characters:

```
char[] chars = {'J', 'a', 'v', 'a'};
```

Conceptually:

```
String  
"Java"
```

versus

```
char[]
```

J	a	v	a
0	1	2	3

A String is immutable, whereas array elements can be changed:

```
chars[0] = 'X';
```

Now:

```
X a v a
```

14. String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable	✗ No	✓ Yes	✓ Yes
Thread-safe synchronization	Immutable	✗ No	✓ Yes
Best for repeated modification	✗	✓	✓
Original Java 1.0	✓	✗	✗

Introduced	Java 1.0	Java 5	Java 1.0

A particularly important historical point is that **StringBuffer was part of the original Java platform**, while `StringBuilder` was added much later in Java 5.

Example:

```
StringBuffer buffer = new StringBuffer("Hello");
```

```
buffer.append(" Java");
```

```
System.out.println(buffer);
```

Output:

Hello Java

15. Real-World Example — User Information

```
public class StringBasics {
    public static void main(String[] args) {
        String firstName = "Anurag";
        String lastName = "Bharti";
        String city = "Bilaspur";

        String fullName =
            firstName + " " + lastName;

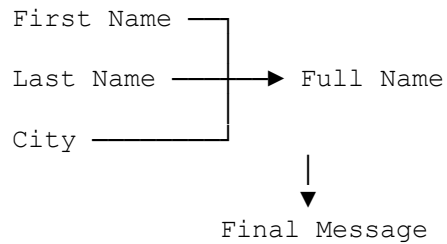
        String message =
            "My name is " + fullName
            + " and I live in " + city + ".";

        System.out.println(message);
    }
}
```

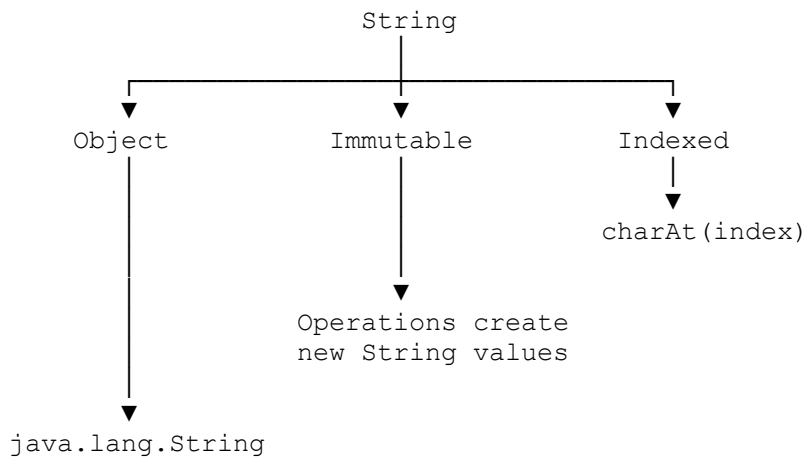
Output:

My name is Anurag Bharti and I live in Bilaspur.

Flow:



16. Important Characteristics of String



Key points

1. String is a class in java.lang.
 2. It represents a sequence of characters.
 3. Strings are immutable.
 4. Indexing starts at 0.
 5. `length()` returns the number of char values.
 6. `==` compares object references.
 7. `equals()` compares string contents.
 8. String literals can be stored in the String pool.
 9. `+` can concatenate strings.
 10. Arrays and strings are different concepts.
 11. StringBuffer was available from the original Java platform for mutable string operations.
-

17. Simple Formula to Remember

String

- Sequence of characters
- Immutable
- Index starts at 0
- `length()` → number of char values
- `charAt(i)` → character at index `i`
- `equals()` → content comparison
- `+` → concatenation

One-line definition

A Java String is an immutable object representing a sequence of characters, with operations for creating, comparing, searching, extracting, and transforming textual data.

Java 1.0 note: `java.lang.String` and the fundamental String language behavior were part of Java's original 1.0 platform in 1996. Modern Java has added many additional String methods and Unicode capabilities over subsequent releases.

Java 1.0 — I/O Streams

Java Version: 1.0

Year: 1996

Feature: I/O Streams

UniqueClassName: `IOStreams`

1. Definition

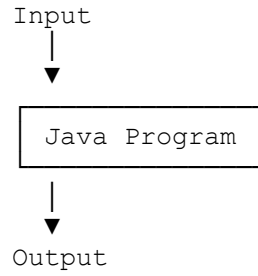
I/O (Input/Output) Streams in Java provide a mechanism for **reading data from a source and writing data to a destination.**

Java's original I/O model is based on streams:

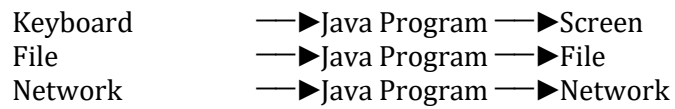
- **Input stream** → reads data **into** a program.
- **Output stream** → writes data **out of** a program.

The original Java I/O classes were primarily provided in the `java.io` package.

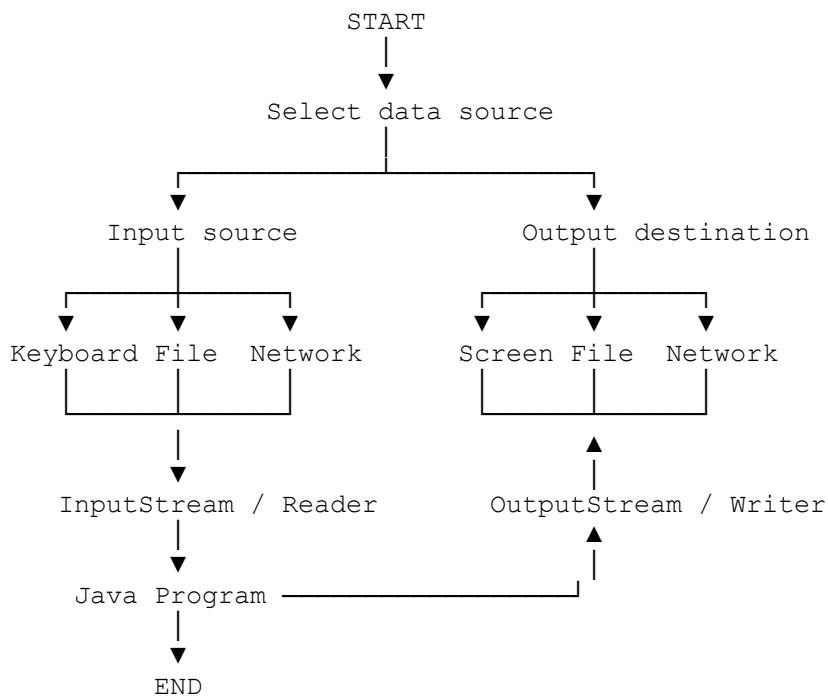
Simple concept



For example:

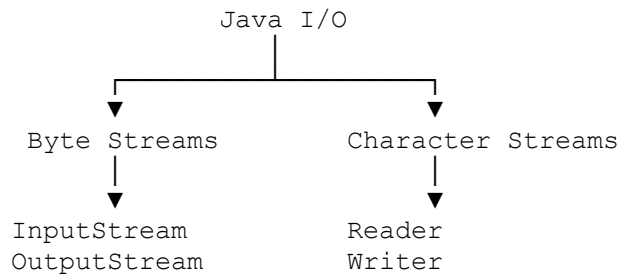


2. I/O Streams Flowchart



3. Two Major I/O Stream Categories

Java I/O streams are broadly divided into:



Byte streams

Used primarily for **binary data** such as:

- images
- audio
- video
- binary files
- raw bytes

Main base classes:

InputStream
OutputStream

Character streams

Used for **character/text data**.

Main base classes:

Reader
Writer

4. Basic Java I/O Code Example

The simplest example uses standard output:

```
public class IOStreams {
```

```
public static void main(String[] args) {  
    System.out.println("Hello, Java I/O Streams!");  
}  
}
```

Output:

Hello, Java I/O Streams!

Here:

System.out

is Java's standard output stream.

5. Reading From Standard Input

Java provides:

System.in

as the standard input stream.

A basic byte-oriented example:

```
import java.io.IOException;  
  
public class IOStreams {  
    public static void main(String[] args)  
        throws IOException {  
  
        System.out.print("Enter a character: ");  
  
        int data = System.in.read();  
  
        System.out.println(  
            "You entered: " + (char) data  
        );  
    }  
}
```

Example

Input:

A

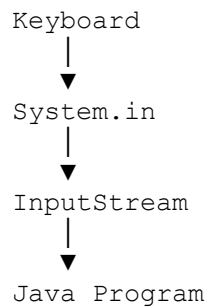
Output:

Enter a character: A

You entered: A

6. How System.in Works

Conceptually:



When you execute:

```
int data = System.in.read();
```

the program reads a byte from the standard input stream.

The return type is int, because `InputStream.read()` uses the value -1 to indicate end-of-stream.

7. Writing to Standard Output

Java provides:

`System.out`

for standard output.

Example:

```
public class IOStreams {  
    public static void main(String[] args) {  
        System.out.write('A');  
    }  
}
```

Output:

A

System.out is a PrintStream, which provides convenient methods such as:

```
print()  
println()  
printf()
```

8. File Input Stream

FileInputStream reads bytes from a file.

Example:

```
import java.io.FileInputStream;  
import java.io.IOException;  
  
public class IOStreams {  
    public static void main(String[] args)  
        throws IOException {  
  
        FileInputStream input =  
            new FileInputStream("input.txt");  
  
        int data;  
  
        while ((data = input.read()) != -1) {  
            System.out.print((char) data);  
        }  
  
        input.close();  
    }  
}
```

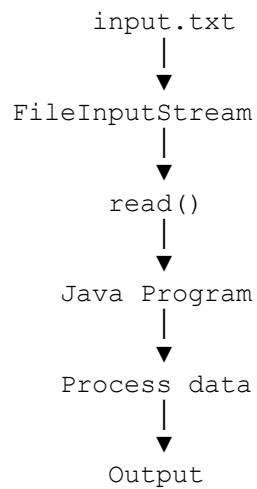
Suppose input.txt contains:

Hello Java
I/O Streams

Output:

Hello Java
I/O Streams

9. File Input Flowchart



10. File Output Stream

FileOutputStream writes bytes to a file.

```
import java.io.FileOutputStream;
import java.io.IOException;

public class IOStreams {

    public static void main(String[] args)
        throws IOException {

        FileOutputStream output =
            new FileOutputStream("output.txt");

        String text = "Hello Java I/O Streams";
```

```

        byte[] data = text.getBytes();

        output.write(data);

        output.close();

        System.out.println("Data written successfully.");
    }
}

```

The file output.txt will contain:

Hello Java I/O Streams

11. File Copy Example

A classic I/O stream example is copying one file into another.

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class IOStreams {

    public static void main(String[] args)
        throws IOException {

        FileInputStream input =
            new FileInputStream("input.txt");

        FileOutputStream output =
            new FileOutputStream("copy.txt");

        int data;

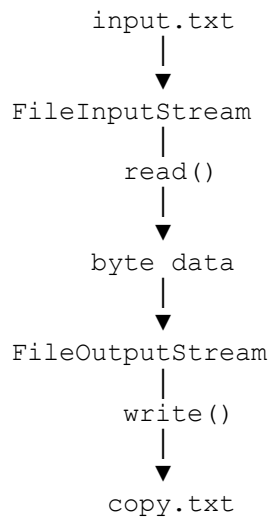
        while ((data = input.read()) != -1) {
            output.write(data);
        }

        input.close();
        output.close();

        System.out.println("File copied successfully.");
    }
}

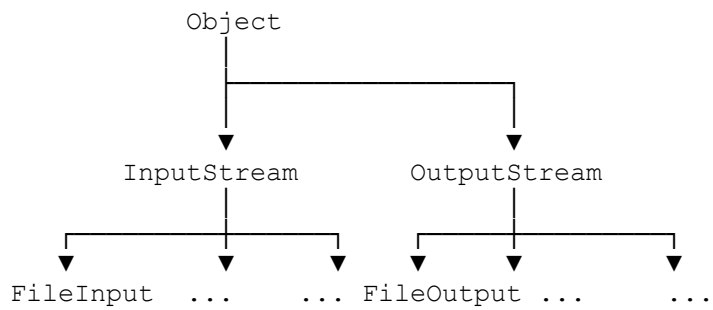
```

Flowchart



12. Byte Stream Hierarchy

The fundamental byte-stream hierarchy is:



Important classes include:

Input

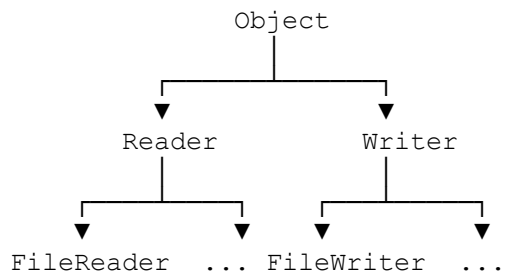
InputStream
FileInputStream
FilterInputStream
BufferedInputStream
DataInputStream

Output

OutputStream
FileOutputStream
FilterOutputStream
BufferedOutputStream
DataOutputStream
PrintStream

13. Character Stream Hierarchy

Character streams use:



Character streams are particularly useful for text.

Example:

```
import java.io.FileReader;
import java.io.IOException;

public class IOStreams {

    public static void main(String[] args)
        throws IOException {

        FileReader reader =
            new FileReader("input.txt");

        int data;

        while ((data = reader.read()) != -1) {
            System.out.print((char) data);
        }

        reader.close();
    }
}
```

14. Byte Streams vs Character Streams

Feature	Byte Stream	Character Stream
Basic units	Bytes	Characters
Main classes	InputStream, OutputStream	Reader, Writer
Typical use	Binary data	Text data
Example	Image	Text file
Original Java I/O model	Yes	Yes
Encoding-aware text abstraction	No	Yes

A useful rule:

Binary data → InputStream / OutputStream
Text data → Reader / Writer

15. Buffered Streams

Reading or writing one byte at a time can be inefficient.

Java provides buffered streams:

BufferedInputStream
BufferedOutputStream

Example:

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class IOStreams {

    public static void main(String[] args)
        throws IOException {

        BufferedInputStream input =
```

```

        new BufferedInputStream(
            new FileInputStream("input.txt")
        );

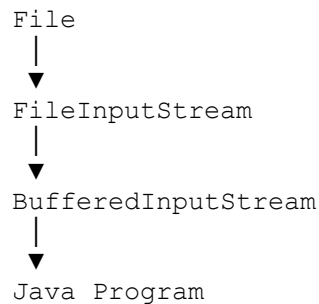
        int data;

        while ((data = input.read()) != -1) {
            System.out.print((char) data);
        }

        input.close();
    }
}

```

Conceptually:



The buffer reduces the need for frequent direct underlying-stream operations.

16. InputStream and OutputStream

InputStream

The fundamental abstract class for byte input.

Common operation:

```
int read()
```

Conceptually:

Source —► InputStream —► Program

OutputStream

The fundamental abstract class for byte output.

Common operation:

`void write(int b)`

Conceptually:

Program —► OutputStream —► Destination

17. Important InputStream Methods

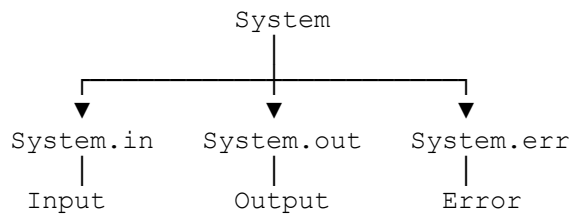
Method	Purpose
<code>read()</code>	Reads one byte
<code>read(byte[])</code>	Reads bytes into an array
<code>read(byte[], int, int)</code>	Reads a portion into an array
<code>available()</code>	Estimates bytes readable without blocking
<code>skip()</code>	Skips bytes
<code>close()</code>	Closes the stream

18. Important OutputStream Methods

Method	Purpose
<code>write(int)</code>	Writes one byte
<code>write(byte[])</code>	Writes a byte array
<code>write(byte[], int, int)</code>	Writes part of a byte array
<code>flush()</code>	Forces buffered output to its destination
<code>close()</code>	Closes the stream

19. Standard Java Streams

Java automatically provides three standard streams:



System.in

Standard input.

`System.in`

System.out

Standard output.

`System.out`

System.err

Standard error output.

`System.err`

Example:

```
public class IOStreams {  
    public static void main(String[] args) {  
        System.out.println("Normal output");  
        System.err.println("Error output");  
    }  
}
```

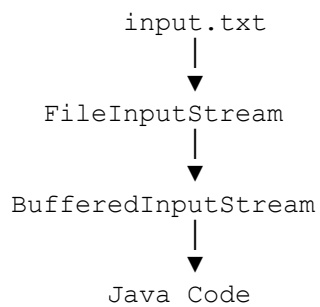
20. I/O Stream Chaining

One of Java I/O's important concepts is **stream composition**.

For example:

```
BufferedInputStream input =  
    new BufferedInputStream(  
        new FileInputStream("input.txt")  
    );
```

Flow:



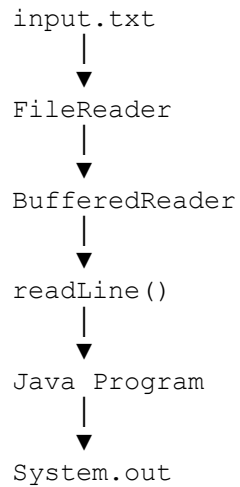
Each layer can provide additional behavior.

21. Real-World Example — Reading a Text File

```
import java.io.BufferedReader;  
import java.io.FileReader;  
import java.io.IOException;  
  
public class IOStreams {  
  
    public static void main(String[] args)  
        throws IOException {  
  
        BufferedReader reader =  
            new BufferedReader(  
                new FileReader("input.txt")  
            );  
  
        String line;  
  
        while ((line = reader.readLine()) != null) {  
            System.out.println(line);  
        }  
    }  
}
```

```
        reader.close();
    }
}
```

Flow:



This is a common pattern for text-file processing.

22. Exception Handling in I/O

Many I/O operations can throw `IOException`.

For example:

```
FileInputStream input =
    new FileInputStream("input.txt");
```

may fail if the file cannot be opened.

One simple approach is:

```
public static void main(String[] args)
    throws IOException {
```

Another is try-catch:

```
import java.io.FileInputStream;
import java.io.IOException;
```

```

public class IOStreams {

    public static void main(String[] args) {

        try {
            FileInputStream input =
                new FileInputStream("input.txt");

            int data;

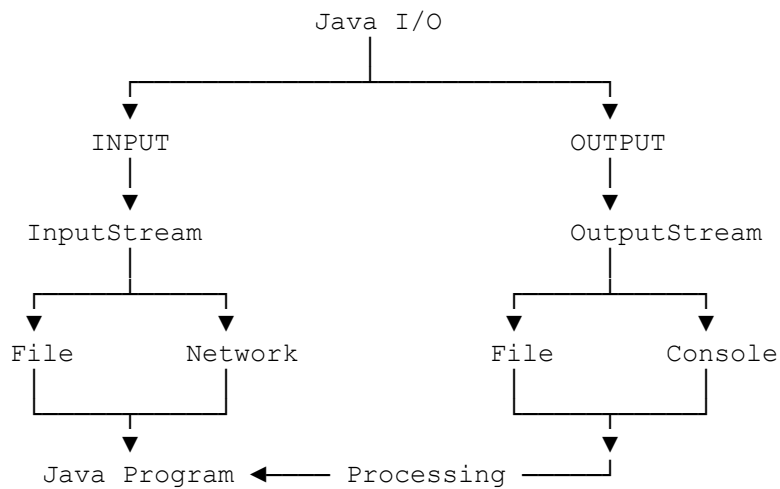
            while ((data = input.read()) != -1) {
                System.out.print((char) data);
            }

            input.close();

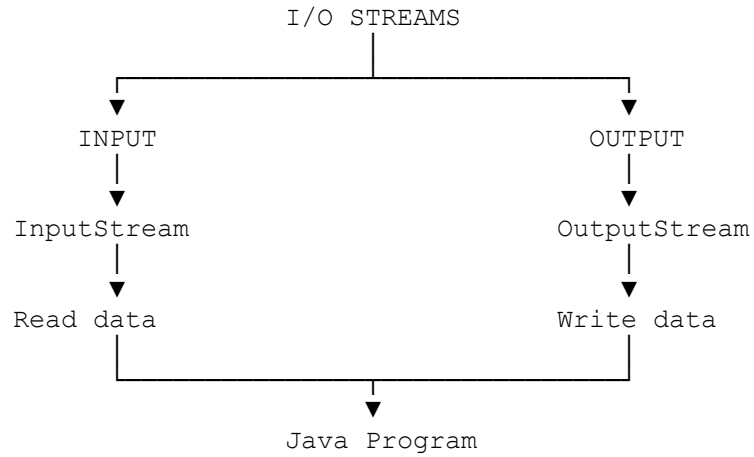
        } catch (IOException e) {
            System.out.println(
                "I/O error: " + e.getMessage()
            );
        }
    }
}

```

23. Complete I/O Flow



24. Key Points to Remember



Important concepts

Concept	Meaning
I/O	Input and Output
Stream	Flow of data between a program and a source/destination
InputStream	Base abstraction for byte input
OutputStream	Base abstraction for byte output
Reader	Base abstraction for character input
Writer	Base abstraction for character output
System.in	Standard input
System.out	Standard output
System.err	Standard error
FileInputStream	Reads bytes from a file
FileOutputStream	Writes bytes to a file
BufferedInputStream	Buffers byte input
BufferedOutputStream	Buffers byte output

FileReader	Reads characters from a file
FileWriter	Writes characters to a file

One-line definition

Java I/O Streams are an abstraction for sequentially reading data from input sources and writing data to output destinations using byte-oriented or character-oriented stream classes.

Java 1.0 note: The core java.io stream architecture—including InputStream, OutputStream, Reader, Writer, file streams, buffered streams, and the standard System.in/out/err streams—formed a fundamental part of Java's original I/O model. Subsequent Java versions added many higher-level I/O APIs, but the original stream model remains foundational.

Java 1.0 — Networking Sockets

Java Version: 1.0

Year: 1996

Feature: Networking Sockets

UniqueClassName: NetworkSockets

1. Definition

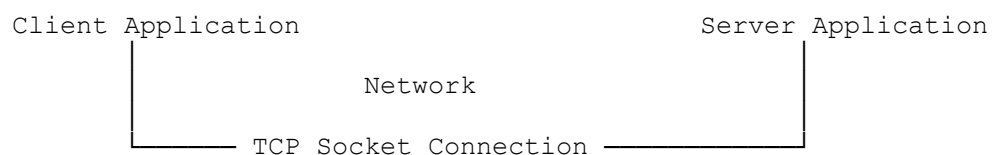
A **network socket** in Java is an endpoint used for communication between two applications over a network.

Java 1.0 introduced networking support through the java.net package, including socket-based communication.

The two fundamental TCP socket classes are:

- Socket — used by the **client** to connect to a server.
- ServerSocket — used by the **server** to listen for incoming TCP connections.

Conceptually:



2. What Is a Socket?

A socket can be thought of as a communication endpoint identified by:

IP Address + Port Number

For example:

192.168.1.10:8080

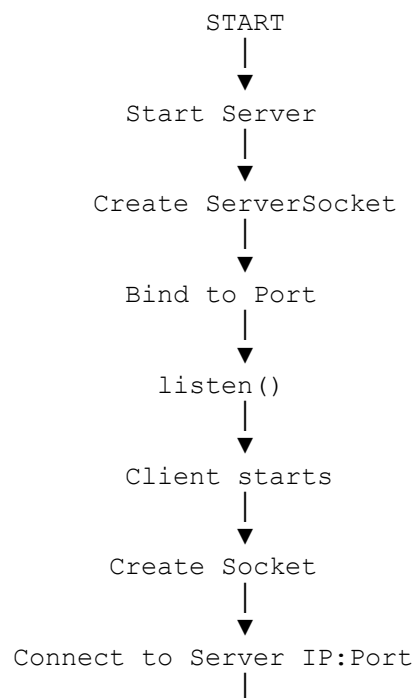
Here:

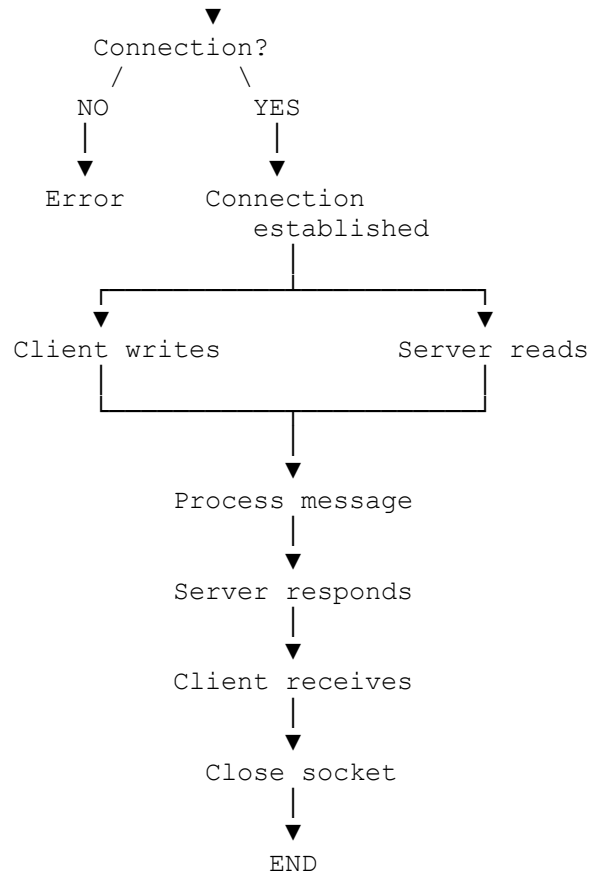
192.168.1.10 → IP address
8080 → Port number

A client connects to the server using the server's IP address and listening port.

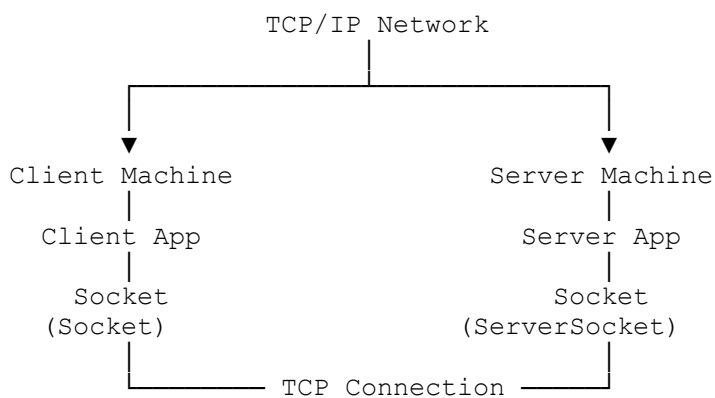
3. Networking Socket Flowchart

TCP Client-Server Communication





4. Basic Socket Architecture



The server generally creates a `ServerSocket`, accepts a connection, and receives a `Socket` representing the connected client.

5. Basic Java Server Example

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class NetworkSockets {

    public static void main(String[] args)
        throws IOException {

        ServerSocket serverSocket =
            new ServerSocket(5000);

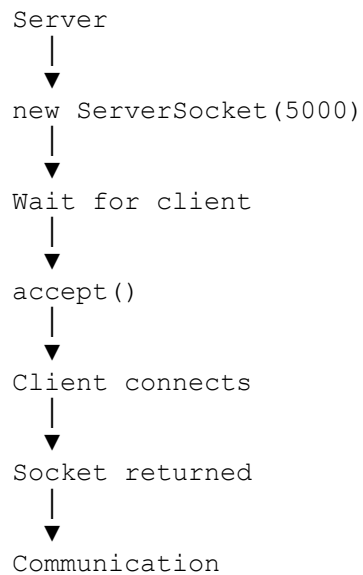
        System.out.println(
            "Server started on port 5000"
        );

        Socket socket = serverSocket.accept();

        System.out.println(
            "Client connected: "
            + socket.getInetAddress()
        );

        socket.close();
        serverSocket.close();
    }
}
```

What happens?



The important operation is:

```
Socket socket = serverSocket.accept();
```

`accept()` waits for an incoming connection.

6. Basic Java Client Example

```
import java.io.IOException;
import java.net.Socket;

public class NetworkSockets {

    public static void main(String[] args)
        throws IOException {

        Socket socket =
            new Socket("localhost", 5000);

        System.out.println(
            "Connected to server"
        );

        socket.close();
    }
}
```

If the server is running on the same computer and listening on port 5000, the client attempts to connect to:

localhost:5000

7. Complete Client-Server Example

The following example demonstrates actual message exchange.

Server

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
```

```
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class NetworkSockets {

    public static void main(String[] args)
        throws IOException {

        ServerSocket serverSocket =
            new ServerSocket(5000);

        System.out.println(
            "Server waiting for client..."
        );

        Socket socket = serverSocket.accept();

        System.out.println(
            "Client connected."
        );

        BufferedReader input =
            new BufferedReader(
                new InputStreamReader(
                    socket.getInputStream()
                )
            );

        PrintWriter output =
            new PrintWriter(
                socket.getOutputStream(),
                true
            );

        String message = input.readLine();

        System.out.println(
            "Client says: " + message
        );

        output.println(
            "Hello Client, message received!"
        );

        input.close();
        output.close();
        socket.close();
        serverSocket.close();
    }
}
```

8. Complete Client

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;

public class NetworkSockets {

    public static void main(String[] args)
        throws IOException {

        Socket socket =
            new Socket("localhost", 5000);

        BufferedReader input =
            new BufferedReader(
                new InputStreamReader(
                    socket.getInputStream()
                )
            );

        PrintWriter output =
            new PrintWriter(
                socket.getOutputStream(),
                true
            );

        output.println(
            "Hello Server!"
        );

        String response = input.readLine();

        System.out.println(
            "Server says: " + response
        );

        input.close();
        output.close();
        socket.close();
    }
}
```

9. Execution Flow

First start the **server**:

Server waiting for client...

Then start the **client**.

Client:

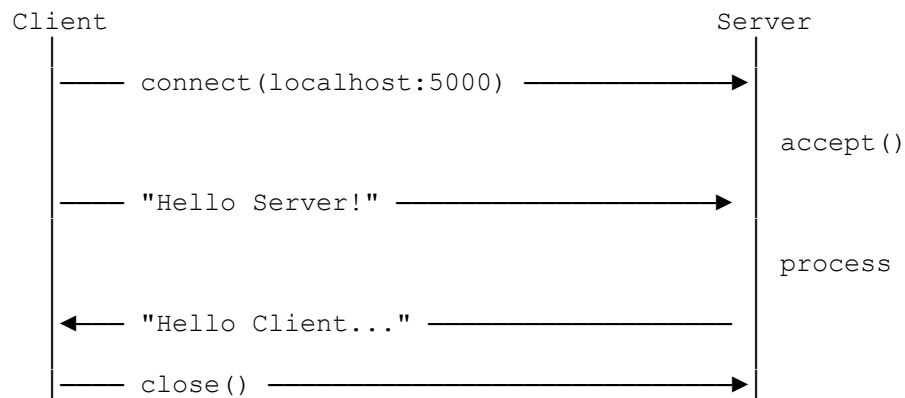
Server says: Hello Client, message received!

Server:

Client connected.

Client says: Hello Server!

Overall flow:



10. ServerSocket vs Socket

Feature	ServerSocket	Socket
Primary purpose	Listen for connections	Communicate over a connection
Typical side	Server	Client and connected server
Creates listening endpoint	Yes	No
Accepts connections	<code>accept()</code>	No
Reads data	No directly	Yes
Writes data	No directly	Yes

Typical constructor	<code>new ServerSocket(port)</code>	<code>new Socket(host, port)</code>
----------------------------	-------------------------------------	-------------------------------------

An important point:

```
ServerSocket serverSocket = new ServerSocket(5000);
Socket socket = serverSocket.accept();
```

The ServerSocket listens, while the returned Socket represents the actual client connection.

11. Socket Input and Output

Once a connection exists:

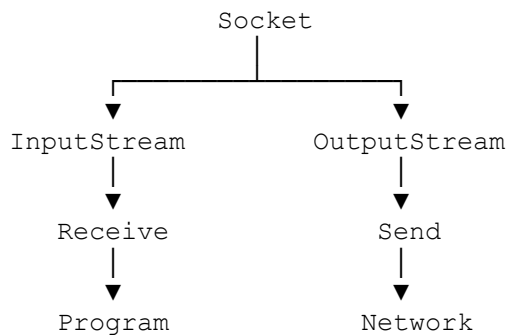
```
socket.getInputStream();
```

provides an input stream.

```
socket.getOutputStream();
```

provides an output stream.

Conceptually:



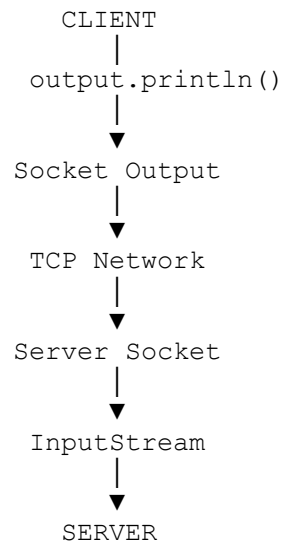
Example:

```
BufferedReader input =
    new BufferedReader(
        new InputStreamReader(
            socket.getInputStream()
        )
    );
```

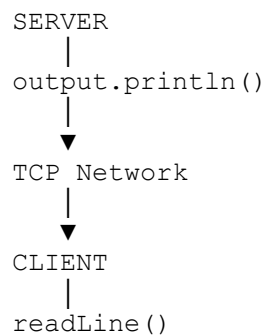
And:

```
PrintWriter output =  
    new PrintWriter(  
        socket.getOutputStream(),  
        true  
    );
```

12. Socket Communication Flow



The reverse direction works similarly:



13. Port Number

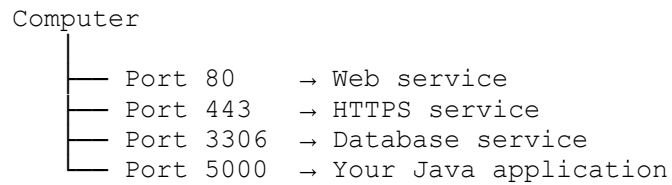
A **port** identifies a network service endpoint on a host.

For example:

```
new ServerSocket(5000);
```

means the server listens on TCP port 5000.

Conceptually:



A port number alone is not enough to identify a remote endpoint; networking normally uses:

IP address + transport protocol + port

14. localhost

In:

```
new Socket("localhost", 5000);
```

localhost means the local machine.

It commonly resolves to a loopback address such as:

127.0.0.1

Therefore:

localhost:5000

means:

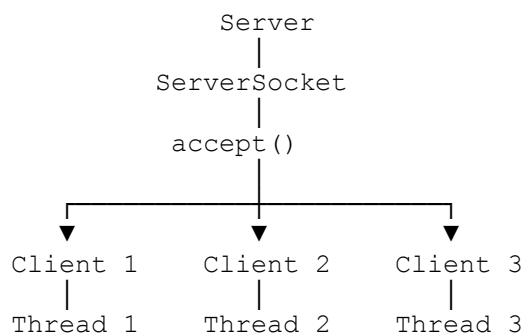
Connect to TCP port 5000 on this same computer.

15. Multiple Clients

A basic server can accept clients one at a time:

```
while (true) {  
    Socket socket = serverSocket.accept();  
  
    // Handle client  
}
```

For multiple simultaneous clients, a common design is to create a separate thread for each connection:



Example:

```
while (true) {  
    Socket socket = serverSocket.accept();  
  
    Thread thread =  
        new Thread(() -> handleClient(socket));  
  
    thread.start();  
}
```

16. Important Socket Exceptions

Network operations commonly involve checked exceptions such as:

IOException
UnknownHostException

For example:

```
try {  
    Socket socket =  
        new Socket("localhost", 5000);  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Possible causes include:

- Server not running
 - Port unavailable
 - Network failure
 - Connection refused
 - DNS/host resolution failure
-

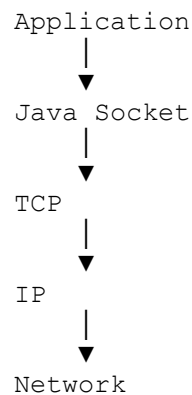
17. TCP Socket Characteristics

Java's Socket/ServerSocket model is primarily based on **TCP**.

TCP provides:

- Connection-oriented communication
- Reliable byte-stream delivery
- Ordered data delivery
- Error detection and retransmission handled by the TCP layer

Conceptually:



18. TCP vs UDP

Java networking also supports UDP through DatagramSocket, which is different from TCP sockets.

Feature	TCP Socket	UDP DatagramSocket
Connection-oriented	Yes	No
Reliable delivery	Yes	No guarantee
Ordered delivery	Yes	No guarantee
Communication model	Byte stream	Datagrams
Main classes	Socket, ServerSocket	DatagramSocket, DatagramPacket

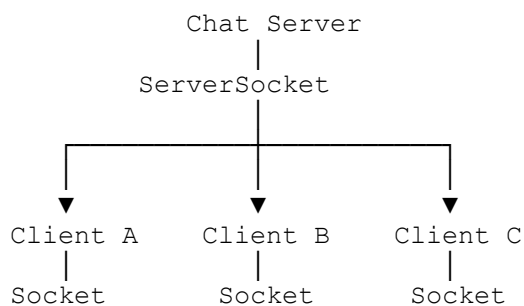
For basic client-server communication:

TCP:
Client ——— connected stream —————> Server

UDP:
Client — packet —> Server
Client — packet —> Server

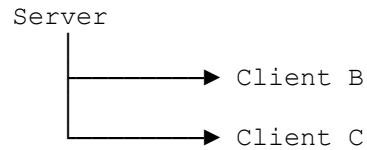
19. Real-World Example

A simple chat system can use sockets:



A message:

Client A
↓
"Hello"



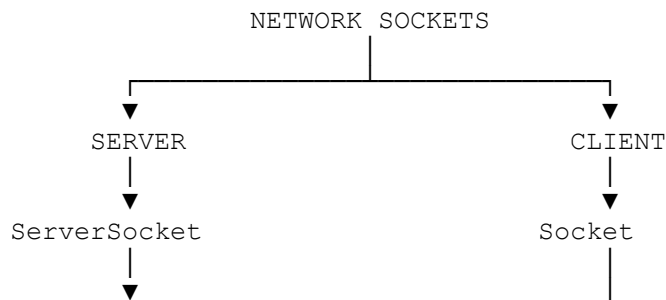
This is the basic architectural idea behind many networked applications, although modern production systems often use higher-level protocols and frameworks.

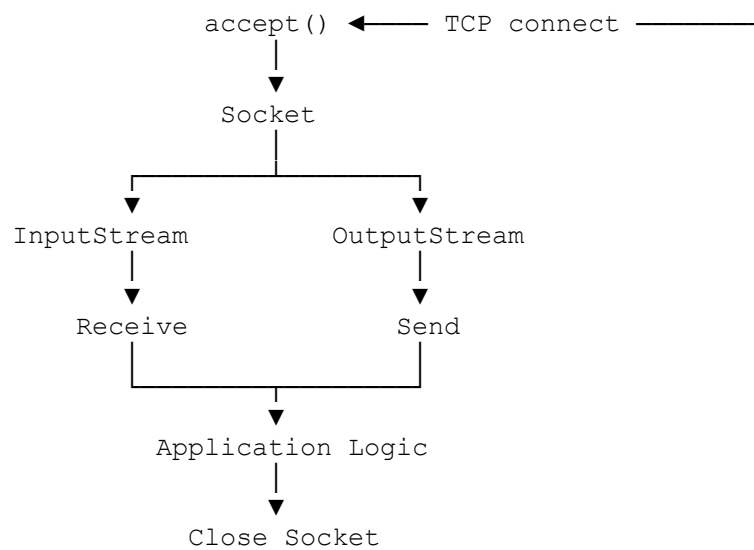
20. Key Java Socket Classes

Class	Purpose
Socket	TCP client/connected endpoint
ServerSocket	TCP server listening endpoint
InetAddress	Represents IP addresses/host information
DatagramSocket	UDP communication
DatagramPacket	UDP packet
URL	Represents a URL
URLConnection	Connection to a URL resource

The first two are the fundamental classes for traditional TCP socket programming.

21. Complete Conceptual Flow





22. Key Points to Remember

1. **Socket** = network communication endpoint.
2. **ServerSocket** = listens for TCP connections.
3. **Socket** = represents a TCP connection endpoint.
4. **accept()** = waits for and accepts an incoming connection.
5. **getInputStream()** = receives bytes.
6. **getOutputStream()** = sends bytes.
7. **Port** = identifies a service endpoint on a host.
8. **TCP sockets** provide connection-oriented byte-stream communication.
9. Multiple clients can be handled concurrently using threads.
10. Socket communication commonly involves `IOException`.

One-line definition

Java networking sockets provide an endpoint-based mechanism for communication between applications over a network, with `Socket` and `ServerSocket` providing the fundamental TCP client-server programming model.

Java 1.0 note: The `java.net` package and core socket networking classes were part of Java's original networking capabilities. The original Java platform included `Socket`, `ServerSocket`, `DatagramSocket`, `DatagramPacket`, and related networking abstractions; subsequent releases expanded the networking APIs considerably.